

SelectOmics

User Guide

A reproducible, leakage-proof feature selector for
small- n , large- p classification

Reference → Cleaning → Regularisation → Wrappers
followed by a mandatory pipeline evaluation that
recommends which step's panel to use

Package version 0.7.1, pre-release

Amanda S. Barnard

September 22, 2026

Contents

1	Introduction	1
1.1	The problem SELECTOMICS solves	1
1.2	The pipeline at a glance	1
1.3	Supported algorithms	2
1.4	Is it right for your data?	3
1.5	Who this guide is for	4
2	Installation	5
2.1	Requirements	5
2.2	Installing	5
2.3	Optional dependency groups	5
3	Quick Start	7
3.1	Minimal Python example	7
3.2	Starting from a preset	7
3.3	Minimal command-line example	8
3.4	What you get back	8
4	Conceptual Overview	9
4.1	Pipeline architecture	9
4.2	The consensus philosophy	9
4.2.1	Bootstrap replicate models	10
4.2.2	Graduated relaxation from unanimity	10
4.2.3	Reading the agreement your panel achieved	10
4.2.4	The terminal fallback	11
4.3	Seed offset strategy	11
5	Input Data and Preprocessing	13
5.1	Supported input formats	13
5.2	Required data structure	13
5.3	From raw file to model-ready matrices	14
5.4	Parallelism and graceful degradation	14
6	Sample-Size Diagnostics	16
6.1	The four signals	16
6.2	Per-step propagation	16

7	The Pipeline Steps in Detail	17
7.1	Step 0: Reference Evaluation	17
7.1.1	Purpose	17
7.1.2	Method	17
7.1.3	Outputs	17
7.2	Step 1: Data Cleaning	17
7.2.1	Purpose	17
7.2.2	Method	18
7.2.3	Class-aware correlation	18
7.2.4	Selection and fallback	19
7.2.5	Outputs	19
7.3	Step 2: Regularisation	19
7.3.1	Purpose	19
7.3.2	Theory	19
7.3.3	Per-algorithm implementation	19
7.3.4	Selecting by rank, not by value	20
7.3.5	Consensus and intersection	20
7.3.6	Terminal or prefilter	21
7.3.7	Outputs	21
7.4	Step 3: Wrappers	21
7.4.1	Purpose	21
7.4.2	When it runs	21
7.4.3	Stage 1: RFECV consensus	21
7.4.4	Stage 2: Stability selection	22
7.4.5	Intersection rule	22
7.4.6	Outputs	23
8	Models and Tuning	24
8.1	Model construction	24
8.2	One randomised tuning pass	24
8.3	Consensus pipelines	24
8.4	Feature importance per model	25
9	Evaluation Metrics	26
9.1	The AUC used throughout	26
9.2	Cross-validation with bootstrap	26
9.3	Comprehensive held-out metrics	26
9.4	Generalisation gap labels	27
10	Pipeline Evaluation	28
10.1	Three validation protocols	28
10.2	The comparison table and adequacy flags	28
10.3	Choosing the panel: the one-standard-error rule	29

10.4 The quality label	29
10.5 The selection-optimism guard	29
10.6 Ranking on held-out folds instead	30
10.7 Retrieving the result	30
11 Visualisations	32
11.1 Pipeline summary	32
11.2 Per-step consensus versus reference	33
11.3 Final model: learning curve and ROC	33
11.4 Final model: per-class ROC and confusion matrix	33
11.5 Validation comparison	35
12 Configuration Reference	36
12.1 Input and output	36
12.2 Algorithm, consensus, and compute	37
12.3 Splitting, cross-validation, and tuning	37
12.4 Step control and evaluation panels	37
12.5 Selection thresholds and targets	38
12.6 Validation, calibration, nested CV, and ranking	39
12.7 Serialisation and construction helpers	39
13 Choosing Key Parameters	41
13.1 <code>n_consensus_models</code>	41
13.2 <code>min_features_floor</code>	41
13.3 <code>min_consensus</code>	42
13.4 Retention targets	42
13.5 If you need sensitivity	42
14 Presets and Suggestions	44
14.1 The omics preset	44
14.2 Data-driven suggestions	45
15 Command-Line Interface	46
15.1 <code>selectomics suggest</code>	46
15.2 <code>selectomics template</code>	46
15.3 <code>selectomics run</code>	46
16 Output Files	48
16.1 Final results	48
16.2 Final figures	49
16.3 Per-step intermediate files	49

17 Advanced Usage	50
17.1 Stepwise and chained execution	50
17.2 Checkpointing and resume	51
17.3 Nested cross-validation	51
17.4 Ranking the steps on held-out folds	52
17.5 Probability calibration	52
17.6 Programmatic logging	52
18 Reproducibility	53
19 Worked Examples	54
20 Benchmarks	55
20.1 The claim, measured	55
20.2 Where it loses, and why that is in this chapter	55
21 Troubleshooting	57
A Configuration Table	59
References	63

List of Figures

11.1	Pipeline summary. Feature count falls from 286 to 16 across the three selection steps while the cross-validation AUC holds at 0.866 against the full-feature reference's 0.868. Left: feature count against CV performance. Right: the features-versus-performance trade-off.	32
11.2	Step 3 consensus versus reference AUC distribution. The consensus of models on 16 features matches the 286-feature reference, which is the result a successful reduction produces.	33
11.3	Final model diagnostics on the 16-feature recommended panel. Left: learning curve with standard-deviation bands. Right: training versus test ROC. The held-out test AUC is 0.858 against a training CV AUC close to it, a gap labelled <i>excellent</i>	34
11.4	Per-class ROC curves (left) and test-set confusion matrix (right) for the final model, fitted on the 16-feature recommended panel and scored on the held-out test set.	34
11.5	Validation comparison across steps. Clockwise from top left: validation method comparison, features versus performance, stability (coefficient of variation, lower is more stable), and bootstrap confidence intervals.	35

Introduction

SELECTOMICS is a Python package that reduces thousands of molecular features to a small, biologically interpretable subset while preserving predictive performance. It is built for the regime that characterises most omics experiments: the number of measured features p far exceeds the number of samples n , and both interpretability and reproducibility matter as much as raw accuracy.

The design goal is narrower than "fewer features". It is to find the *right* features, and to be able to say how strongly the evidence supports them. A LASSO fit returns a short list too; on the benchmark suite it returns 38 features of which about half are real. SELECTOMICS returns 8 of which almost all are real (precision 0.970 against 0.501), and reports the level of model agreement that panel holds at.

1.1 The problem SelectOmics solves

High-throughput omics assays (transcriptomics, methylation, copy-number, microRNA, proteomics) routinely produce feature matrices with hundreds to tens of thousands of columns but only tens to low hundreds of rows. In this small- n , large- p setting, classifiers overfit easily, individual feature rankings are unstable, and a feature set selected from one train/test split often fails to reproduce on another. SELECTOMICS addresses this through four design commitments:

- **Progressive, multi-criterion reduction.** Rather than trusting a single selection method, SELECTOMICS applies three conceptually distinct selection stages in sequence. Each stage removes features that fail a different, increasingly sophisticated criterion, so a feature must be defensible from several independent points of view to survive.
- **Consensus over replicates.** Within each stage, the decision to keep or drop a feature is not made by one model fit. It is made by a vote across many independently seeded models trained on bootstrap-resampled data.
- **Agreement is reported, not requested.** Every step begins by demanding unanimity and relaxes only as far as it must to reach the feature floor. What it reports is the strongest level of agreement at which your panel actually holds.
- **Nothing is scored on data that chose it.** The test split is removed before any selection runs and is touched exactly once, at the end, on the recommended panel. Selection, validation, and the recommendation all live inside the training split.

1.2 The pipeline at a glance

A reference evaluation fixes the performance baseline, three selection stages run in order, and a mandatory pipeline evaluation decides which of their outputs to deliver:

1. **Step 0, Reference.** No selection. A single tuned model is cross-validated on the full feature set, producing the baseline AUC, its per-fold distribution, and a box plot every later step is compared against.
2. **Step 1, Data cleaning.** Removes constant, low-variance, and redundant features, by consensus between a variance filter and a correlation filter.
3. **Step 2, Regularisation.** Runs parallel L1-style and L2-style regularised models and keeps features that both penalties agree are informative.
4. **Step 3, Wrappers.** Intersects recursive feature elimination with cross-validation (RFECV) and stability selection.
5. **Pipeline evaluation (mandatory).** Every step's panel is re-validated under three independent resampling protocols, and one of them is *recommended* for downstream use.
6. **Feature profile evaluation (optional).** Per-step comparisons against the Step 0 reference, provenance, and diagnostic figures.
7. **Model evaluation (optional).** The selected algorithm, refit on the recommended panel alone, scored on the held-out test set, which no selection step, evaluation, or training pass has seen.

The step you asked for is not always the step you get

The pipeline evaluation can recommend an *earlier* step than the last one that ran, and on real data it does. On the bundled ovarian microRNA example, Step 3 cuts Step 2's 115 features to 10 and the recommendation keeps the 115. This is the reason to enable every step rather than stopping early: a step that prunes past the point where it helps costs you nothing, because the evaluation catches it. Retrieve the delivered panel with `get_recommended_features()`.

What makes a feature “survive”

At every stage the guiding principle is agreement. Two independent criteria must concur, and a consensus of independently seeded models must vote to keep the feature. This is what gives SELECTOMICS its stability relative to single-method selection.

1.3 Supported algorithms

Four classifiers are supported throughout the pipeline, selected with the `algorithm` option:

Key	Model	Primary strength
LR	Logistic Regression	Interpretable linear boundary; fast
XGB	XGBoost gradient boosting	Non-linearity; native handling of missing values (default)
RF	Random Forest	Non-parametric; captures feature interactions
SVM	Support Vector Machine (RBF)	Effective in high-dimensional, low-sample settings

XGB is the default. If the input contains missing values and the configured algorithm is LR or SVM, the algorithm is automatically switched to XGB, because XGBoost handles missing entries natively without imputation. RF is left unchanged for the same reason.

The same data, run several ways

Nothing in the pipeline is specific to one learner. Re-running the same configuration with a different `algorithm` is a legitimate and cheap robustness check: features that survive under both a linear and a tree model carry a stronger claim than features that survive under one. Expect the reported agreement level to differ between algorithms, for the reason given in Chapter 4.

1.4 Is it right for your data?

Use SELECTOMICS when a short, defensible panel is the deliverable. Measured on a pre-registered synthetic benchmark and on real TCGA data, against LASSO, ElasticNet, RFECV, SHAP, RF importance and permutation importance at their library defaults (Chapter 20):

- **Precision 0.970**, against 0.501 for LASSO, 0.555 for SHAP and 0.232 for ElasticNet. When it returns a feature, that feature is almost always real.
- **It does not invent signal.** On a null control with no informative features at all, it returns 5.8 false positives. LASSO returns 53.6, ElasticNet 180.
- **The lead grows with p** , which is the regime it is built for: F1 0.915 at $p \geq 5000$, against 0.553 for the next-best method.
- **On real data it concedes almost nothing.** 18 features from 34 021 on TCGA lower-grade glioma, at AUC 0.9885 against the best method's 0.9891: a panel 31 times shorter than LASSO's for 0.0006 AUC.

Do not use it when

- **Sensitivity is the objective.** Recall is 0.730 against LASSO's 0.893. It will miss true features. `allow_union_rung=True` buys recall back and pays for it in precision (Chapter 13).
- **Runtime is binding.** About 100 s per trial against seconds for LASSO, and close to two hours per cross-validation fold on 34 000 features.
- **n is large and the signal dense.** There a default ElasticNet beats it.

Nor should you expect it to be the most stable method on real data. It leads on stability on synthetic data, but on real TCGA data ElasticNet and LASSO return more consistent panels on every layer measured. The advantage that survives contact with real data is parsimony, not stability.

1.5 Who this guide is for

This guide is written for researchers and analysts applying SELECTOMICS to real datasets. It covers installation, a quick start, the theory and implementation of every stage, the recommendation logic, the full configuration reference, the command-line interface, the output files, the diagnostic figures, the benchmark results, and advanced workflows such as checkpoint/resume and nested cross-validation. No prior knowledge of the package internals is assumed, but familiarity with supervised classification and cross-validation is helpful.

Three companion documents go further in specific directions: the Methods Guide (`docs/METHODS_GUIDE.md`) gives the algorithmic detail behind each step, the Interpretation Guide (`docs/INTERPRETATION_GUIDE.md`) reads every output file and figure panel line by line, and the benchmark report (`benchmarks/BENCHMARKS.md`) carries the measurements this guide quotes.

Installation

2.1 Requirements

SELECTOMICS requires Python 3.9 or newer, and is tested on 3.9 through 3.13. Its core runtime dependencies are installed automatically:

Package	Minimum	Why that floor
numpy	1.24	
pandas	1.5	
scikit-learn	1.4	First release whose Random Forest accepts NaN inputs
scipy	1.9	
xgboost	2.0	Introduced the <code>device=</code> argument used throughout
matplotlib	3.6	
openpyxl	3.0	Excel input and output
pyarrow	8.0	Parquet and Feather input and output
pyyaml	6.0	YAML configs, which the CLI advertises

2.2 Installing

SELECTOMICS is a pre-release and is not yet on PyPI. Install it from GitHub:

```
pip install git+https://github.com/amaxiom/SelectOmics.git
```

To run the examples or the test suite, work from a clone instead, as an editable development install:

```
git clone https://github.com/amaxiom/SelectOmics.git
cd SelectOmics
pip install -e ".[dev,viz]"
```

The examples are Jupyter notebooks that download their data on first run, so they also need Jupyter and a network connection. The package itself needs neither.

2.3 Optional dependency groups

Three optional extras add capabilities that are not needed for the core pipeline:

Group	Adds	When you need it
[viz]	seaborn $\geq$ 0.11	Enhanced confusion-matrix heatmaps. Without it the confusion matrix falls back to a plain <code>imshow</code> .
[io]	tables $\geq$ 3.7, fastparquet $\geq$ 0.8	Required for HDF5 I/O. Parquet already works through <code>pyarrow</code> .
[dev]	pytest $\geq$ 7, pytest-cov $\geq$ 4	Running the test suite. Not needed for analysis.

```

pip install "SelectOmics[viz] @ git+https://github.com/amaxiom/SelectOmics.git"
pip install -e ".[viz,io]"                # from a clone, groups combined

```

GPU acceleration

If a CUDA-capable GPU and `xgboost` 2.0 or newer are present, `SELECTOMICS` detects the GPU and passes `device='cuda'` to `XGBoost` automatically. Detection is a safe probe: if it fails for any reason the pipeline falls back to CPU silently. Set `use_gpu=False` to disable the probe. Note that this is the one setting that trades reproducibility for speed, because the CUDA implementation finds splits differently; leave it off for runs you intend to compare against CPU-generated numbers.

Quick Start

3.1 Minimal Python example

The pipeline is driven by a plain dictionary of options passed to `SelectOmicsPipeline`. Only `data_path` and `target_column` are required.

```
import SelectOmics as selectomics

# Activate logging so you can see pipeline progress (optional).
selectomics.enable_logging()

config = {
    "data_path": "data/omics.csv",
    "target_column": "Class",
}

pipeline = selectomics.SelectOmicsPipeline(config)

# All three selection steps, the pipeline evaluation, and the held-out test.
results = pipeline.run()

# The panel the evaluation recommends. Usually the last step's, and an
# earlier step's when a later one pruned past the point where it helped.
features = pipeline.get_recommended_features()
print(f"Recommended {len(features)} features.")

# The last step's panel, whatever the evaluation said.
last_step = pipeline.get_selected_features()

# Persist config, both feature lists, tables, metrics, and figures.
pipeline.save_results()
```

Use `get_recommended_features()`

`get_selected_features()` returns whichever step ran last. That is the right answer most of the time and the wrong answer exactly when it matters, because a step that over-pruned still ran last. Unless you specifically want a named step's output, ask for the recommended panel.

3.2 Starting from a preset

Instead of hand-setting parameters, start from a preset builder and override only what you need.

```
from SelectOmics import SelectOmicsConfig

cfg = SelectOmicsConfig.standard("data/omics.csv", "Class")
cfg = SelectOmicsConfig.quick("data/omics.csv", "Class", algorithm="RF")

pipeline = selectomics.SelectOmicsPipeline(cfg)
results = pipeline.run()
```

If you are unsure which settings suit an unfamiliar dataset, ask the package:

```
cfg = SelectOmicsConfig.suggest("data/omics.csv", "Class")
```

`suggest` reads the file, measures shape, missingness and class balance, sets defaults to match, and logs its reasoning (Chapter 14).

3.3 Minimal command-line example

The same run can be launched from a terminal via the `selectomics` command installed with the package:

```
selectomics run data/omics.csv --target Class --preset standard --output results/
```

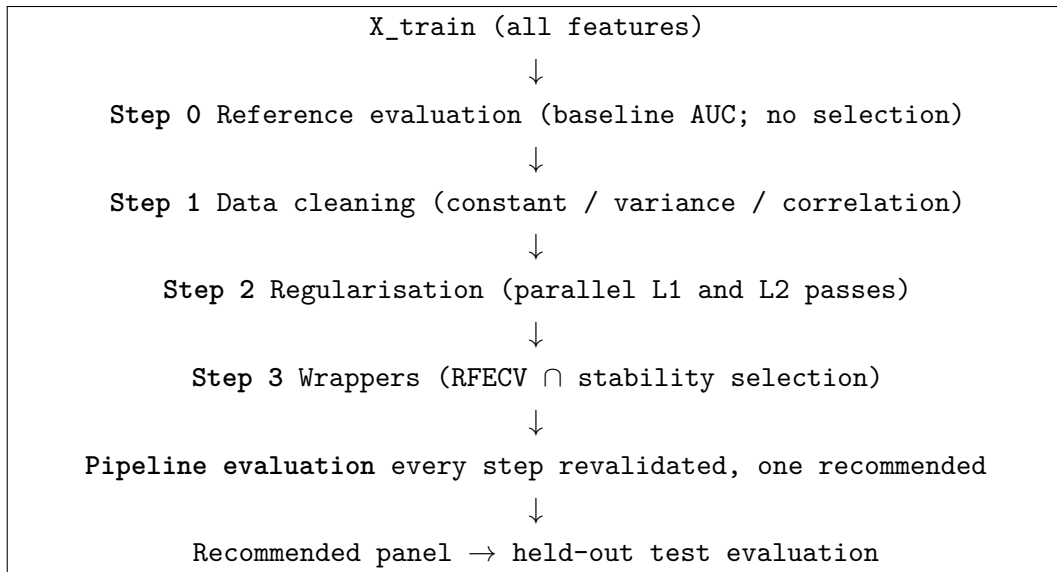
3.4 What you get back

A completed run produces, in `output_dir`: the last step's feature list, the recommended feature list, a per-step summary of feature counts and cross-validation AUC, the multi-protocol validation comparison with its recommendation, the full set of held-out test metrics, per-feature provenance across the selection stages, and a collection of diagnostic figures. These are described in full in Chapter 16.

Conceptual Overview

4.1 Pipeline architecture

SELECTOMICS is organised as a linear sequence of stages that operate on a shrinking feature matrix. The training data is split off once at the start; the held-out test set is never touched by any selection stage, by the validation, or by the recommendation.



Several architectural choices are shared by every stage:

- **A single shared cross-validation splitter.** One `StratifiedKFold` object is created in `load_data` and reused by every step, so fold assignments are identical across all evaluations and comparisons are apples-to-apples.
- **One tuning pass.** The active algorithm is tuned once, before Step 0, by a randomised hyperparameter search. The tuned settings are then locked and reused (with only the random seed varying) to build the consensus models in every stage.
- **An explicit reference baseline.** Step 0 computes a single-model cross-validated AUC on the full feature set. This number is cached and passed to every later stage so that each stage can report whether reduction has helped, hurt, or left performance unchanged.
- **Pass-through when disabled.** Any of Steps 1 to 3 can be switched off. A disabled step passes its input matrix and the reference baseline through unchanged, and the recommendation attributes it correctly rather than crediting it with the reference score.

4.2 The consensus philosophy

The defining feature of SELECTOMICS is that no single model fit decides anything. Two ideas combine to produce stable selections.

4.2.1 Bootstrap replicate models

Within a selection stage, m models (`n_consensus_models`) are built from the tuned pipeline, differing only in random seed. When these models are evaluated, each fold's training split is bootstrap-resampled (sampled with replacement, preserving class balance) with a unique per-(model, fold) seed. The held-out fold is never resampled. This injects genuine diversity: each replicate sees a slightly different view of the data, so features that only look important on one particular arrangement of the data do not survive.

4.2.2 Graduated relaxation from unanimity

Steps 2 and 3 each pair two independent stages and intersect them: a feature survives when *every* stage agrees, and a stage agrees when at least **required** of its m replicates voted for it.

required *always starts at m* , unanimity, and is lowered one vote at a time until the intersection yields `min_features_floor` features, stopping early if the next rung would fall below `min_consensus`. Three consequences are worth stating plainly.

- **The panel size is the input; the agreement level is the output.** They trade against each other and the data decides where the trade lands, so you specify one and observe the other. This is why there is no parameter for “how conservative should the pipeline be”.
- **votes_required is the strongest level your panel holds at.** Because the descent always begins at unanimity, the reported level is a property of the data rather than of a setting.
- **The floor is capped at what the votes support.** The most an intersection can return is what survives a single vote in every stage. Asking for more would guarantee padding with features no model selected, so the request is capped at that ceiling and the capping is logged.

Relaxing gradually rather than falling from intersection straight to union preserves information. With two stages, a union *is* the vote requirement at its floor, so a direct fallback jumps from the strictest setting to the loosest in one move and discards every level between. At $m = 10$ there are ten rungs; that scheme used two.

4.2.3 Reading the agreement your panel achieved

Every step returns `votes_required`, `agreement` ($= \text{votes_required}/m$), `agreement_label`, and `consensus_outcome`.

Achieved	Label	How to read it
100%	unanimous	every model agreed on every feature
80 to 99%	strong	a solid claim
60 to 79%	moderate	defensible, worth stating alongside the panel
40 to 59%	weak	the models disagree substantially
below 40%	minimal	the panel size was reached by relaxing almost all the way
fallback	none (ranked, not agreed)	no consensus existed; features ranked by importance only

```
step2 = pipeline.results["step2"]
print(f"{step2['n_features_out']} features at "
      f"{step2['votes_required']}/{step2['n_models']} agreement "
      f"({step2['agreement_label']})")
```

Expect this to vary by algorithm

On the same data at 5 models, Random Forest typically settles at **strong** while XGBoost and SVM reach only **minimal**, because RF gives non-zero importance to far more features (247 to 696 against XGBoost's roughly 22) and so has a much wider pool to agree about. A **minimal** label is not necessarily a bad result, but it is a weaker claim, and it belongs in any write-up alongside the feature list.

4.2.4 The terminal fallback

Rank-averaging is reachable only when no feature was selected by any replicate in every stage, so there is no consensus evidence to aggregate at all. It ranks within the union of features that received a vote somewhere, stepping outside that pool only if it cannot fill the request. Firing this fallback earlier was measured and rejected: at equal panel size, voting matched or beat ranking at every agreement level from 0.1 to 1.0.

4.3 Seed offset strategy

Reproducibility is exact. Every stochastic operation is derived from the single `random_seed`. To guarantee that models in different stages never accidentally share a seed, each stage uses a distinct seed spacing:

Context	Seed formula
Step 1 consensus evaluation	<code>base + i*100</code>
Step 2 L1 voting	<code>base + i*200</code>
Step 2 L2 voting	<code>base + i*200 + 100</code>
Steps 2/3 consensus evaluation	<code>base + i*100</code>
Step 3 RFECV	<code>base + i*400</code>
Step 3 stability subsample	<code>base + i*500</code>
Step 3 stability model	<code>base + (i mod m)*400 + 500</code>
Consensus factory	<code>base + i*100</code>
Bootstrap (per fold)	<code>model_seed + fold*1000</code>

Given the same `random_seed`, every model in every step receives the same seed on every run, and no global NumPy random state is modified.

Input Data and Preprocessing

5.1 Supported input formats

SELECTOMICS reads a single table in which one column is the class label and the remaining columns are features. The following formats are supported, with the format auto-detected from the file extension unless `input_format` is set explicitly:

Format	Extensions
CSV	.csv
TSV	.tsv, .txt
Excel	.xlsx, .xls (sheet via <code>excel_sheet_name</code>)
Parquet	.parquet, .pq
HDF5	.h5, .hdf5 (dataset key via <code>hdf5_key</code>)
Feather	.feather
JSON	.json (records orientation)

Set `input_format` explicitly when the extension is missing or non-standard. A format the package does not support raises `ValueError` before the read is attempted; anything a reader itself raises, on a corrupt or mislabelled file, is wrapped in a `RuntimeError` naming both the path and the format attempted.

5.2 Required data structure

- **Rows** are samples (observations, patients, specimens).
- **Columns** are features, plus one target column, which may appear anywhere.
- The target column holds class labels, strings or integers. Binary and multiclass problems are both supported; at least two classes are required.

```
SampleID, GeneA, GeneB, GeneC, Class
S001, 1.23, 0.45, 2.11, Control
S002, 0.89, 1.67, 0.34, Case
S003, 2.01, 0.22, 1.88, Case
```

A non-numeric identifier such as `SampleID` is dropped automatically, because only numeric feature columns are retained. A *numeric* identifier column would be treated as a feature, so remove it before running.

5.3 From raw file to model-ready matrices

Loading and preparation happen in `load_data`, in a fixed order:

1. **Load.** The file is read into a pandas DataFrame.
2. **Separate target and features.** The `target_column` is extracted as y ; the remaining columns become X . Only numeric feature columns are retained. The feature index is sorted for reproducible ordering.
3. **Handle missing values.** The number of NaNs is counted. If any are present and the chosen algorithm is LR or SVM (which cannot accept NaNs), the algorithm is automatically switched to XGB and a warning is logged regardless of `verbose`. For XGB and RF, NaNs are deliberately preserved so the tree learner can treat missingness as signal. No imputation is ever performed.
4. **Encode labels and split.** Labels are integer-encoded with `LabelEncoder`. A single stratified train/test split is created with `test_size` (default 0.25) and `random_seed`.
5. **Choose cross-validation folds.** If `cv_splits` is not set, the fold count is chosen automatically as

$$n_{\text{splits}} = \min(\text{max_cv_splits}, \max(\text{min_cv_splits}, c_{\min})),$$

where $c_{\min}$ is the size of the smallest class in the training set. This guarantees every fold can contain at least one sample of every class. A single `StratifiedKFold` object is built and shared across the whole pipeline.

6. **Tune the active model.** The chosen algorithm is tuned by randomised search (Chapter 8) and the resulting pipeline is cached for reuse.

Scaling

Every model pipeline begins with `MinMaxScaler`, which scales each feature to $[0, 1]$. The scaler is fitted only on the training portion within each cross-validation fold, so no information leaks from validation folds or the held-out test set into the scaling.

5.4 Parallelism and graceful degradation

SELECTOMICS uses every available core by default and needs nothing installed to do so. It degrades rather than failing when the machine cannot cooperate: no CUDA GPU means XGBoost runs on CPU silently, one usable core means it runs serially, and a matrix too small for threading to pay off is fitted on a single thread.

Thread count does not change results

A run at `n_jobs=-1` selects exactly the features a run at `n_jobs=1` selects, so `n_jobs` is purely a speed-versus-courtesy dial. Set it to 1 when you are launching several pipelines concurrently yourself. This guarantee required pinning XGBoost to `tree_method='exact'`: its default `hist` draws subsample rows in a thread-order-dependent way, so the same seed produced different trees at different thread counts. The pinned method is also several times faster in the small- n regime this package targets.

Sample-Size Diagnostics

Before Step 0 runs, SELECTOMICS assesses whether the dataset is large enough to support reliable feature selection and performance estimation. The assessment (`assess_sample_size_adequacy`) produces warnings but never halts the run: the intent is to make the user aware of statistical limitations, not to prevent analysis.

6.1 The four signals

Each signal is flagged *adequate*, *caution*, or *critical*, with thresholds drawn from the methodological literature.

Signal	Critical	Caution	Source
Absolute count n	$n < 30$	$30 \leq n < 100$	Varoquaux (2018)
Ratio n/p	$n/p < 0.1$	$0.1 \leq n/p < 1.0$	Varoquaux (2018)
Samples per class	any class < 10	any class 10 to 20	Figuerola et al. (2012)
CV fold reliability	$c_{\min} < k$	$k \leq c_{\min} < 2k$	Kohavi (1995)

Here $c_{\min}$ is the smallest class size and k is the number of cross-validation folds. The overall severity reported is the worst flag across all four signals.

Expect warnings in this regime

An omics dataset in the regime this package targets will trigger these signals, often more than one. That is the point of printing them: they qualify the result rather than invalidate it. What they mean in practice is that the validation confidence intervals will be wide and the held-out test AUC will carry real uncertainty, so read Chapter 10 with that in mind.

6.2 Per-step propagation

The n/p signal is re-evaluated at the start of every step using the current feature count. Because feature selection lowers p , a dataset that starts in the *caution* band for n/p may improve to *adequate* as features are removed. Warnings are printed only when severity is not *adequate*, and *critical* warnings are never suppressed regardless of verbosity.

The Pipeline Steps in Detail

This chapter describes each stage: its purpose, the theory behind its method, the algorithm as implemented, the per-algorithm specifics, and the files it writes.

7.1 Step 0: Reference Evaluation

7.1.1 Purpose

Step 0 performs no selection. It establishes the classification AUC baseline on the full, unfiltered training set. This baseline is cached and handed to every later step, so each step can be judged on whether it preserves performance while reducing dimensionality.

7.1.2 Method

A single tuned model (never an ensemble, regardless of `n_consensus_models`) is evaluated by stratified cross-validation on all features. Two constraints are deliberate:

- **Single model.** The reference is one model registered as `Ref-<algorithm>` (for example `Ref-XGB`), so the baseline is a fair single-model estimate against which single consensus models are compared.
- **No bootstrap.** The reference uses `use_bootstrap=False`: it must reflect the unmodified training distribution. Bootstrap diversity is reserved for the consensus stages.

The primary metric is the per-fold macro one-vs-rest AUC. Step 0 also assembles an out-of-fold probability matrix (one held-out probability estimate per sample), used later for plotting and downstream analysis.

7.1.3 Outputs

Step 0 returns the reference result: the mean and standard deviation of the fold AUCs, the per-fold AUCs themselves, and the out-of-fold probabilities and predictions. When intermediate saving and visualisation are enabled it also writes `step0_reference_boxplots`, `step0_reference_roc`, and `step0_summary.txt`. The box plot of per-fold scores is the reference every later step's box plot is drawn against.

7.2 Step 1: Data Cleaning

7.2.1 Purpose

Step 1 removes features that are uninformative by construction: those that are constant, those with negligible variance, and those that are redundant because they duplicate a more informative feature.

7.2.2 Method

The step proceeds in three sub-steps.

1a. Constant removal. Any feature with at most one distinct value in the training set is dropped immediately. Such features carry no information and would distort the correlation matrix.

1b/1c. Joint variance and correlation threshold search. Rather than tuning a variance filter and a correlation filter separately, SELECTOMICS searches an 11×11 grid of threshold pairs and selects the pair at which the two filters agree most strongly on what to remove.

- The **variance grid** is 11 evenly spaced values from 0 to `max_variance_threshold`. A feature is dropped if its variance is at or below the threshold.
- The **correlation grid** is 11 evenly spaced values from `min_correlation_threshold` to 1.0, where a threshold of 1.0 keeps every feature. The correlation matrix is computed once and reused. For each threshold, a greedy pass processes features in descending priority order and, for every pair correlating above the threshold, drops the lower-priority member. This is fully deterministic and independent of column order.
- The **objective** maximised over the grid is the Jaccard similarity of the two drop sets,

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

where A is the variance drop set and B is the correlation drop set. Normalising by the union prevents aggressive thresholds from gaming the objective by producing large but incoherent drop sets; it rewards genuine agreement.

7.2.3 Class-aware correlation

Which correlation matrix the filter uses, and which member of a correlated pair it keeps, is governed by `class_aware_correlation` (default `True`):

Setting	Matrix	Priority	Tie-breaks
True	pooled within-class	correlation ratio η^2	variance, then name
False	total absolute Pearson	variance	name

Why class-aware is the default

Features carrying the same class signal are mutually correlated *because* they are informative. Total correlation cannot distinguish that from redundant duplication, so it keeps one carrier and discards the rest. Centring each feature on its own class mean removes the between-class covariance, leaving only association the label does not explain: genuine duplicates stay correlated after centring, co-regulated markers do not. On the synthetic case built to expose this, retention of 20 informative features went from 1 of 20 to 20 of 20.

One consequence is worth stating. A feature with no within-class variation, one perfectly determined by the label, centres to all zeros and so is never judged redundant. That is the intended trade-off rather than an oversight, since such a feature is maximally informative. Two identical class-determined probes therefore both survive Step 1.

7.2.4 Selection and fallback

The kept set is the intersection of the two best kept masks. If that falls below `min_features_floor` it falls back to the union, and if the union is still too small, to the top `min_features_floor` features by variance.

Step 1 has only two voters, the variance filter and the correlation filter, and each casts a single binary vote. Unlike Steps 2 and 3 there are therefore no intermediate rungs to relax through: intersection and union are the entire ladder. It reports its outcome in the same vocabulary regardless, as `votes_required` out of `n_models=2` with an `agreement_label`, so a panel decided here carries the same record as one decided later.

7.2.5 Outputs

Step 1 writes the cleaned matrix and the individual drop lists: `step1_X_clean`, `step1_dropped_by_constant`, `step1_dropped_by_variance`, `step1_dropped_by_correlation`, `step1_dropped_by_consensus` (features both filters flagged), `step1_summary.txt`, and the consensus-versus-reference box plot and ROC figures. The return dictionary also records the best variance and correlation thresholds and the Jaccard score they achieved.

7.3 Step 2: Regularisation

7.3.1 Purpose

Step 2 removes features that regularised models can shrink away without losing predictive power. It runs an L1-style (sparsity-inducing) pass and an L2-style (smoothing) pass in parallel and keeps only features that both passes consider informative.

7.3.2 Theory

L1 (Lasso-style) regularisation penalises the sum of absolute weights and drives uninformative coefficients exactly to zero, producing sparse solutions. L2 (Ridge-style) regularisation penalises the sum of squared weights and shrinks all coefficients smoothly toward zero, preferentially suppressing small noisy weights. Running both means a feature must survive under both a sparsity assumption and a smoothing assumption, so the result does not depend on either assumption being correct.

7.3.3 Per-algorithm implementation

The abstract L1/L2 distinction is realised differently for each algorithm:

Algorithm	L1 stage	L2 stage
LR	penalty='l1', saga, C=0.1	penalty='l2', C=0.1
XGB	reg_alpha=10.0, reg_lambda=0.1	reg_alpha=0.1, reg_lambda=10.0
RF	50 trees, max_features='sqrt', min_samples_leaf=3	100 trees, max_features='log2', min_samples_leaf=2
SVM	LinearSVC, l1, C=0.1	LinearSVC, l2, C=1.0

For Random Forest, tree sparsity is the L1/L2 analogue: fewer, leafier trees concentrate importance (like L1), while more, deeper trees diffuse it (like L2). Because the tuned RBF SVM exposes no usable coefficients, a `LinearSVC` surrogate supplies importances for the SVM case; the tuned RBF SVM is still used for all AUC scoring. Importance is extracted as the maximum absolute coefficient across classes for linear models, and as the built-in gain or impurity importance for XGB and RF. All importances are normalised to $[0, 1]$.

7.3.4 Selecting by rank, not by value

Each stage keeps the highest-importance features, with the count set by a binary search over the full $[0, 1]$ percentile domain (at most 12 evaluations) targeting `step2_target_retention`.

Why rank and not a percentile of the values

Tree importance is exactly zero for every feature never used in a split, and that tie mass is 98 to 99.8% of the array on this package's target data, so `np.percentile` returns 0.0 across the whole search range and the kept count is identical at every point. Selection therefore works on importance *rank*, capped at the number of features the model gave a non-zero importance to. The cap is what keeps this honest at the other end: a feature the model never split on carries no evidence for keeping it, so selection never extends into the tied-zero region even when the retention target asks for more. When the cap binds, the step logs it and raises a `UserWarning` naming the target and the closest achievable value.

`step2_target_retention` is a fraction of the reachable pool

Its default of 1.0 keeps the whole reachable pool per stage, where the pool is the set of features the stage models actually used. On wide data that pool is around 1% of the input: measured, one XGB stage fit used 21 of 2000 features and 30 of 10 000. Because the L1 and L2 stages are then intersected, Step 2 as a whole returns fewer. The pool size itself is set by `step2_stage_colsample`, and *lowering* it widens the pool, because at `colsample` 1.0 every tree sees every feature and keeps splitting on the same strongest few.

7.3.5 Consensus and intersection

At the chosen ranks, m independently seeded models vote in each stage (L1 seeds `base + i*200`; L2 seeds `base + i*200 + 100`). The final Step 2 set is the intersection of the two stages, taken at the strongest vote level that yields `min_features_floor` features, by the graduated relaxation

of Chapter 4. Rank-averaging over the stage models' mean importance is the terminal fallback, reachable only when no feature was selected by any replicate in both stages.

7.3.6 Terminal or prefilter

`step2_role` (default `"terminal"`) decides what Step 2 is for. As a terminal step it makes its own final decision and hands Step 3 a panel that is already nearly pure signal. As a `"prefilter"` it deliberately keeps more, leaving the real cut to Step 3. The prefilter role reaches the same recall as the sensitivity switch of Chapter 13 but takes roughly nine times the runtime to do it, which is why it is available rather than default.

7.3.7 Outputs

Step 2 writes `step2_X_reg_clean`, `step2_dropped_features`, `step2_summary.txt`, and the consensus-versus-reference figures. The return dictionary records the best L1 and L2 percentiles, the per-stage selection masks, and the achieved agreement.

7.4 Step 3: Wrappers

7.4.1 Purpose

Step 3 uses two wrapper-based methods that judge features by their effect on a fitted model, and keeps only features that both methods select.

7.4.2 When it runs

Step 3 is skipped, passing its input through unchanged, when `enable_step3` is `False` or when fewer than `MIN_FEATURES_FOR_RFECV= 30` features reach it.

Why the gate is at 30

The gate stands in for a quantity that cannot be observed at runtime: how much noise the incoming panel still holds. Recursive elimination is only useful on a panel that still contains features worth eliminating, and Step 2's output does not, its precision being 0.970 across the benchmark suite. Lowering the gate to 10 was tried and reverted: on three measured runs Step 2 delivered a panel of 10 features at F1 1.000 and Step 3 cut it to one or two, dropping F1 to 0.182. A wide incoming panel is the available proxy for an unfiltered one.

The practical consequence is that on very wide inputs, where Step 2 cuts hard, the shipped pipeline is effectively two steps. Step 3 fires in 1 of 55 benchmark runs. It remains valuable standalone, and after a deliberately permissive Step 2 (`step2_role="prefilter"`).

7.4.3 Stage 1: RFECV consensus

Recursive Feature Elimination with Cross-Validation repeatedly fits a model, ranks features by importance, and removes the least important, tracking the cross-validated score at each size. The feature-count floor, `min_features_to_select`, is set by binary search over the full $[0, 1]$

percentile domain to target `step3_target_retention` of the incoming features (default 60%). To keep the inner search fast, RFECV uses an internal `StratifiedKFold` with $\min(3, k)$ folds, scored by `roc_auc_ovr`. The estimator per algorithm is:

Algorithm	RFECV estimator (importance source)
LR	Logistic Regression, lbfgs (max absolute coefficient)
XGB	XGBoost (gain importance)
RF	Random Forest, 50 trees (impurity importance)
SVM	<code>LinearSVC</code> , l1, C=0.1 surrogate (max absolute coefficient)

The RBF SVM has no per-feature coefficients and permutation importance inside RFECV would be prohibitively expensive, so a sparse L1 `LinearSVC` surrogate supplies the importances; the tuned RBF SVM is still used for all AUC scoring. Across m RFECV runs (seeds spaced 400 apart) the selection masks are summed, giving each feature a vote count.

7.4.4 Stage 2: Stability selection

Stability selection (Meinshausen and Bühlmann, 2010) measures how consistently a feature is chosen across random subsamples of the data. SELECTOMICS draws 50 stratified subsamples (the standard lower bound recommended by Shah and Samworth, 2013), each containing 50% of the training samples without replacement. On each subsample a model is fit, importances are extracted, and a feature counts as selected on that subsample if its importance exceeds the subsample mean (a scale-invariant, threshold-free criterion). The selection frequency of a feature is the fraction of valid subsamples in which it was selected. Subsamples missing a class are skipped; if all subsamples fail, stability selection abstains rather than vetoes, retaining every feature so the intersection reduces to the RFECV result.

7.4.5 Intersection rule

The Step 3 set is the intersection of the RFECV and stability selections, taken at the strongest vote level that yields `min_features_floor` features. Stability frequencies are first mapped onto the vote scale, as $\lfloor \text{freq} \times m \rfloor$ for features clearing `stability_threshold` and zero otherwise, so the stability stage relaxes in step with the model votes rather than acting as a fixed veto.

stability_threshold is a floor, not a target

Its declared default is `None`, which resolves to 0.6, the permissive end of the 0.6 to 0.9 range Meinshausen and Bühlmann give theoretical guarantees over. Below 0.6 the bound is no stronger than a single model run. At 0.9 the filter demands near-unanimity across 50 subsamples, which measured far stricter than unanimity across 5 or 10 models: on one development scenario every threshold at or above 0.7 selected nothing at all. Features above the floor still face the full vote requirement and relax with it.

7.4.6 Outputs

Step 3 writes `step3_X_wrappers_final`, `step3_dropped_features`, `step3_selection_diagnostics` (per-feature RFECV votes and fractions, stability frequency, per-stage and final masks), `step3_summary.txt`, and the consensus-versus-reference figures.

Models and Hyperparameter Tuning

8.1 Model construction

Every model is a two-step scikit-learn pipeline: `MinMaxScaler` followed by the classifier. This is true of the tuned reference model, the consensus replicates, the stage models, the validation models, and the final model.

8.2 One randomised tuning pass

Before Step 0, the active algorithm is tuned once by `RandomizedSearchCV` with `quick_tune_`
`iterations` candidates (scoring `roc_auc` for binary problems and `roc_auc_ovr` for multiclass). The tuning cross-validation uses between 2 and 5 folds, chosen from the smallest class size, and runs single-threaded for deterministic behaviour. Only the selected algorithm is searched; the other three receive fixed-default pipelines used for reference comparison only.

The search spaces are:

Algorithm	Search space
LR	<code>C~LogUniform(0.01, 100)</code> ; <code>penalty=l2</code> ; <code>solver</code> ∈{ <code>lbfgs</code> , <code>saga</code> }
XGB	<code>n_estimators~Randint(50, 300)</code> ; <code>max_depth~Randint(3, 8)</code> ; <code>learning_rate~LogUniform(0.01, 0.3)</code> ; <code>subsample</code> , <code>colsample_bytree~Uniform(0.6, 1.0)</code> ; <code>reg_alpha</code> , <code>reg_lambda~LogUniform(0.01, 10)</code>
RF	<code>n_estimators~Randint(50, 300)</code> ; <code>max_depth</code> ∈{ <code>None</code> }∪ <code>Randint(3,20)</code> ; <code>min_samples_split~Randint(2,10)</code> ; <code>min_samples_leaf~Randint(1,5)</code> ; <code>max_features</code> ∈{ <code>sqrt</code> , <code>log2</code> , <code>None</code> }; <code>class_weight=balanced</code>
SVM	<code>C~LogUniform(0.1, 100)</code> ; <code>gamma~LogUniform(10⁻⁴, ^{10⁻¹)}; <code>kernel=rbf</code>; <code>probability=True</code>; <code>class_weight=balanced</code></code>

8.3 Consensus pipelines

Consensus replicates are built from the tuned model by locking its tuned hyperparameters and varying only `random_state`. This is what makes the consensus vote a measure of stability under

resampling rather than under retuning. For XGBoost, the GPU `device` setting is inherited from the tuned pipeline.

8.4 Feature importance per model

Algorithm	Importance
LR	max absolute coefficient across classes
XGB	gain-based <code>feature_importances_</code>
RF	mean decrease in impurity (<code>feature_importances_</code>)
SVM	max absolute coefficient of a <code>LinearSVC</code> surrogate (RBF SVM has none)

The pipeline delivers a set, not an ordering

Consensus selection answers whether a feature is in or out. It does not produce a defensible ranking of the survivors, and none is reported as though it were one. If you need an ordering of the delivered panel, fit your own model on it and compute SHAP values or permutation importance there, where the ordering can be attributed to a specific model on specific data.

Evaluation Metrics

9.1 The AUC used throughout

The primary metric across every step is the macro one-vs-rest AUC. For more than two classes, each class is scored against the rest and the per-class AUCs are averaged with equal weight, which is robust to class imbalance. SELECTOMICS uses a custom implementation that skips classes absent from a given fold (returning 0.5 if no class can be scored), avoiding the errors scikit-learn would raise on incomplete folds.

9.2 Cross-validation with bootstrap

In the consensus stages, each fold's training split is bootstrap-resampled (seed `model_seed + fold*1000`, stratified) before fitting; the held-out fold is never resampled. The evaluator returns per-fold AUCs, their mean and standard deviation (`ddof=1`, and 0.0 for a single fold), and an out-of-fold probability matrix assembled by writing each fold's held-out predictions into the corresponding rows, so every sample receives exactly one probability estimate.

9.3 Comprehensive held-out metrics

At the end of the run, the final model is refit on the *recommended* features and evaluated on the untouched test set. `final_<ALGO>_comprehensive_metrics.json` records, where `<ALGO>` is the algorithm actually used:

- Overall accuracy and balanced accuracy; macro and weighted precision, recall, and F1.
- Training cross-validation AUC (mean and standard deviation), test macro AUC, and their difference, the generalisation gap.
- The step whose panel was scored, its name and feature count, and the agreement level that panel achieved.
- For each class: precision, recall, F1, support, and AUC.
- For each class, three operating points: the default argmax sensitivity and specificity; the Youden's-J optimal threshold with its sensitivity and specificity; and the sensitivity achievable at fixed 95% and 99% specificity.

How the high-specificity thresholds are chosen

For a target specificity, SELECTOMICS considers only operating points whose false positive rate is at or below $1 - \text{specificity}$, then picks the most sensitive of those. This guarantees the reported operating point genuinely meets the requested specificity, rather than landing just short of it.

9.4 Generalisation gap labels

The train-to-test AUC gap is summarised with a plain-language label: *excellent* (gap < 0.05), *good* (< 0.10), *moderate* (< 0.15), or *poor* otherwise.

Read the gap, not only the AUC

This is the one number in the run computed on samples that nothing in the pipeline has seen, and it is the only check on optimism that every panel shares. On the bundled ovarian copy-number example the recommended panel validated at well on the training split and scored 0.584 on the held-out test, a gap of 0.226 labelled *poor*. Nothing computable from the training split alone could have caught that, because the reference panel was optimistic there too. A poor gap means the panel should not be used, whatever its validation score said.

Pipeline Evaluation and the Recommendation

After the selection stages complete, `validate_features` subjects the feature set produced at *each* step to an independent, multi-protocol assessment on the training split only. This answers a different question from the per-step consensus AUC: how does each candidate panel hold up under several distinct resampling schemes, and which one should be delivered?

This evaluation is not optional in any meaningful sense. It is what turns a sequence of selection steps into a recommendation, and `run()` performs it unless you explicitly pass `validate=False`.

10.1 Three validation protocols

The `FeatureSetValidator` runs three protocols per feature set:

1. **Stratified k-fold cross-validation**, repeated over three model seeds, scored by `roc_auc_ovr`. Reports mean and standard deviation.
2. **Leave-one-out or stratified shuffle-split**. Full leave-one-out is used when $n \leq 50$; otherwise a 30-iteration stratified shuffle-split with an adaptively sized test fraction.
3. **Bootstrap out-of-bag**. Repeated bootstrap resampling (`n_bootstrap`, default 100) with evaluation on the out-of-bag samples, producing a 95% confidence interval from the 2.5th and 97.5th percentiles.

A protocol that loses most of its iterations still returns a number, so both loops count their failures and warn when fewer than half the iterations completed. Read such a warning as saying the interval rests on that many samples, not on the number you configured.

10.2 The comparison table and adequacy flags

The results are collated into `feature_set_validation`, one row per step, with the cross-validation, leave-one-out, and bootstrap AUCs and standard deviations, the bootstrap confidence interval and its width, and two adequacy flags. A confidence-interval width above 0.3 is flagged *critical*, above 0.2 *caution*. The `overall_adequacy` column combines this with the up-front sample-size severity, so that a narrow interval on an underpowered dataset is not read as reassurance.

Each candidate is summarised by a weighted composite AUC,

$$\text{weighted} = 0.5 \text{ cv} + 0.25 \text{ loo} + 0.25 \text{ bootstrap},$$

which is the score the steps are ranked on by default.

10.3 Choosing the panel: the one-standard-error rule

The top-scoring panel is not simply taken. SELECTOMICS applies the classical one-standard-error rule: among all panels scoring within one standard error of the best, it delivers the *smallest*, breaking size ties by score.

Why argmax is the wrong rule here

Taking the highest score directly was measured and is actively harmful. On one development scenario it took a 620-feature panel at F1 0.032 over an 18-feature one at F1 0.714, for an AUC difference of 0.0003. Across 12 runs argmax never improved on the last step and halved mean F1 twice, because AUC and selection quality pull in opposite directions at $n \ll p$.

The tolerance is the widest defensible uncertainty estimate available, the largest of the cross-validation standard error, the bootstrap standard deviation, and the confidence-interval half width, *floored* at `MIN_SCORE_TOLERANCE= 0.005`.

Why the rule needs a floor

AUC saturates at $n \ll p$. Measured on one development scenario, every panel from 18 features to the full 2000 scored a cross-validation AUC of exactly 1.0000 with a standard deviation of 0.0000, so a variance-derived band had zero width and the rule collapsed back to argmax exactly where it was needed most. Half a point of AUC is below anything that changes a downstream decision, and no difference below that can justify hundreds of extra features, whatever the variance estimates say.

When parsimony rather than raw score decided the choice, the reported **reason** says so explicitly and names both panels, so a reader comparing the verdict against the comparison table can see that the winner not topping it was deliberate.

10.4 The quality label

The chosen panel is labelled from the score it was ranked on and its stability ratio (standard deviation over mean):

Label	Condition
RECOMMENDED	score above 0.8 and stability ratio below 0.15
CAUTION	score above 0.7
NOT RECOMMENDED	otherwise

10.5 The selection-optimism guard

Those thresholds are applied to a score that selection has already inflated, because validation cross-validates each panel on the same samples its features were selected from. The more aggressive the selection, the more optimistic the score. On a null control with no informative

features at all, a panel validated at 0.84 and would have been labelled **CAUTION** or better while its held-out AUC was 0.46.

The least-selected panel, Step 0 with every feature, carries no selection at all and is therefore the one unbiased reference in the comparison. When the recommended panel beats it by more than `MAX_PLAUSIBLE_SELECTION_GAIN= 0.05`, the label is downgraded and the reason says why.

Where 0.05 comes from

Measured with nested cross-validation on GBM, OV and LGG microRNA cohorts and a 5000-feature synthetic set, 20 outer folds in all, the recommended panel's validation score exceeded the reference's by at most 0.021. On the null control, where selection can only fit noise, it exceeded it by 0.23 to 0.37. A gain above 0.05 cannot be told apart from selection fitting noise using the training split alone, so it is flagged rather than trusted.

The guard changes the *label* only, never the choice, and reports `reference_id`, `selection_gain`, and `selection_optimism_suspected` so you can see what it saw.

What the guard cannot catch

It compares the panel against Step 0. When Step 0 is itself optimistic, as it was on the bundled ovarian copy-number example, the gain is small and the guard stays quiet while every panel is inflated together. Optimism shared by every panel is what the held-out test exists to expose, and no training-split statistic substitutes for it.

10.6 Ranking on held-out folds instead

By default the ranking uses validation that shares its samples with selection. Setting `enable_holdout_ranking=True` instead ranks the steps on `holdout_ranking_splits` folds carved out of the *training* split, so no panel is scored on the rows that selected it. The test split is untouched either way. The recommendation then reports `ranking_basis` as `'held-out folds from the training split'`, takes its tolerance band from the spread of the fold means, derives the quality label from that score, and disables the optimism guard, which has nothing left to correct.

Measured, and off by default

This sounds strictly better and is not. An AUC ranking at $n \ll p$ turns on noise and favours larger panels. On seven datasets at five folds it changed three of seven choices and every change was worse on ground truth: a multiclass F1 of 0.99 became 0.18, a rescue case went from 0.73 to 0.24, and on the null control it selected 592 noise features. The AUC differences it acted on were 0.002 to 0.025. Switch it on when predictive AUC on new samples is the deliverable rather than a short panel. Section 8 of the benchmark report carries the per-dataset numbers.

10.7 Retrieving the result

```
# Present only when a recommendation could be built, so use .get().
```

```
rec = pipeline.results.get("recommendation")
if rec is not None:
    print(rec["step_id"], rec["n_features"], rec["quality"])
    print(rec["ranking_basis"], rec["ranking_score"])
    if rec["selection_optimism_suspected"]:
        print("gain over", rec["reference_id"], "=", rec["selection_gain"])

# results["validation"] is a different thing: the comparison table and the
# raw per-protocol results, not the choice made from them.
comparison = pipeline.results["validation"]["comparison_df"]

features = pipeline.get_recommended_features()
```

Visualisations

When `create_visualizations` is enabled, SELECTOMICS writes a set of diagnostic figures in `plot_format` (png, svg, pdf, tiff, or jpeg; raster figures at 300 dpi, JPEG at quality 95). All figures use the viridis colour map, place legends clearly, and are designed to be read without reference to the console log.

Example data provenance

The figures in this chapter come from the bundled quickstart run: a five-class microRNA classification of the TCGA Glioblastoma Multiforme (GBM) cohort (244 samples, 286 microRNA features), algorithm RF, 5 consensus models, seed 42. It is reproducible in about four minutes from `examples/GS-GBM/GBM_Quickstart.ipynb`. The data derive from The Cancer Genome Atlas (TCGA) Research Network and were obtained through the NCI Genomic Data Commons. See the References for citations.

11.1 Pipeline summary

The pipeline summary (Figure 11.1) is the headline diagnostic. The left panel overlays the feature count (falling sharply) against the cross-validation AUC (roughly flat or rising) across the steps; the right panel shows the same as a features-versus-performance trade-off. Together they make the central claim of the method visible at a glance: dimensionality collapses while predictive performance is preserved.

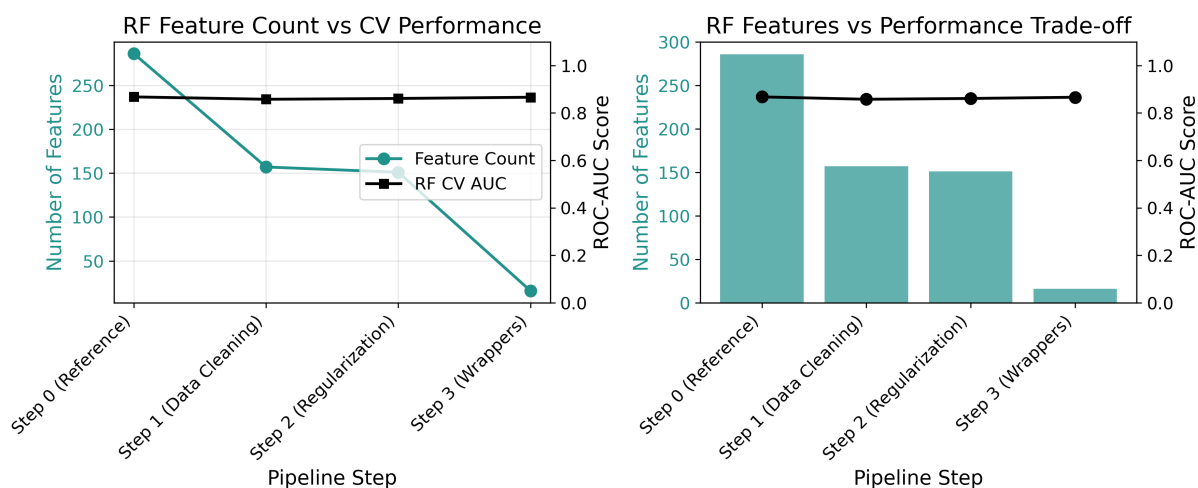


Figure 11.1: Pipeline summary. Feature count falls from 286 to 16 across the three selection steps while the cross-validation AUC holds at 0.866 against the full-feature reference’s 0.868. Left: feature count against CV performance. Right: the features-versus-performance trade-off.

11.2 Per-step consensus versus reference

Each step emits a box plot and an ROC figure comparing its consensus models against the Step 0 reference (Figure 11.2). The box plot shows the distribution of per-fold AUCs; overlapping boxes at or above the reference indicate that the reduction did not cost performance. This is the figure that makes the per-step impact of selection legible, and it is why Step 0 exists.

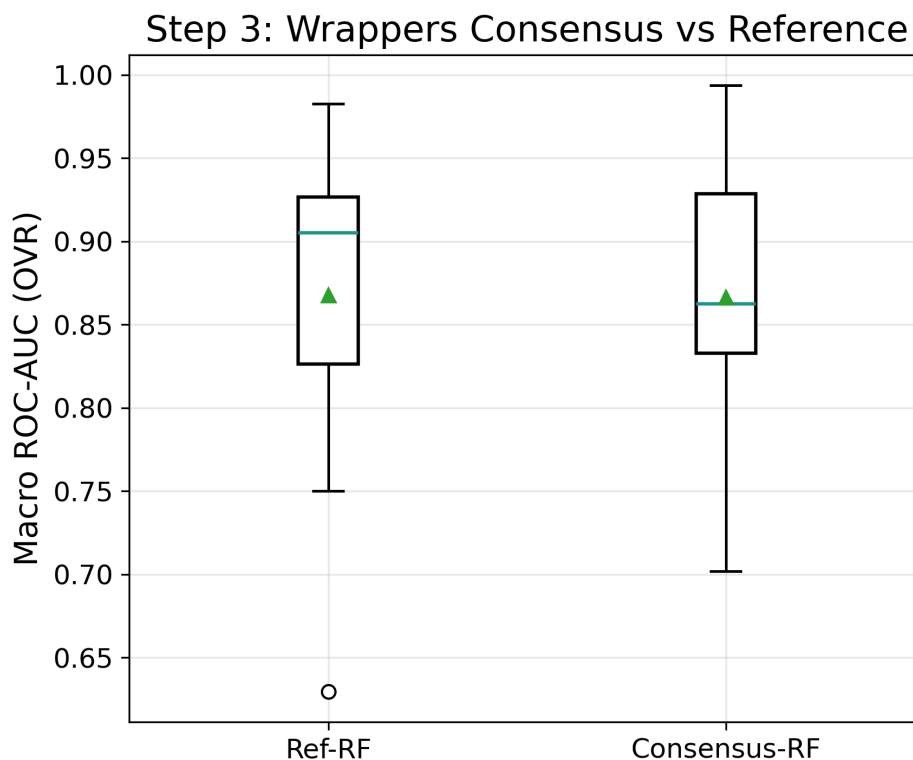


Figure 11.2: Step 3 consensus versus reference AUC distribution. The consensus of models on 16 features matches the 286-feature reference, which is the result a successful reduction produces.

11.3 Final model: learning curve and ROC

The final-model figure (Figure 11.3) pairs a learning curve with a training-versus-test ROC. The learning curve shows training and cross-validation AUC as a function of training-set size, with shaded standard deviation bands; a converging gap suggests the model is not starved of data. The ROC panel overlays the training cross-validation ROC and the held-out test ROC, with a marker at the 95% specificity operating point. The separation between those two curves is the generalisation gap in visual form.

11.4 Final model: per-class ROC and confusion matrix

Figure 11.4 shows one ROC curve per class with its AUC, and the test-set confusion matrix as a heatmap. This is where per-class strengths and weaknesses become visible: a class with a high AUC but a confused row indicates a threshold problem, not a ranking problem.

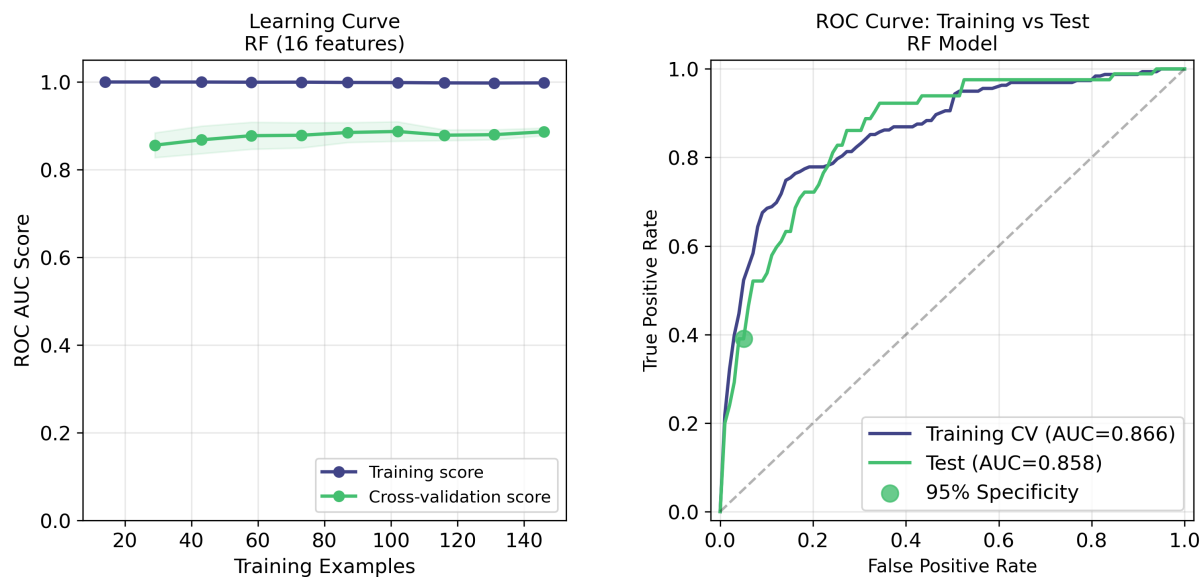


Figure 11.3: Final model diagnostics on the 16-feature recommended panel. Left: learning curve with standard-deviation bands. Right: training versus test ROC. The held-out test AUC is 0.858 against a training CV AUC close to it, a gap labelled *excellent*.

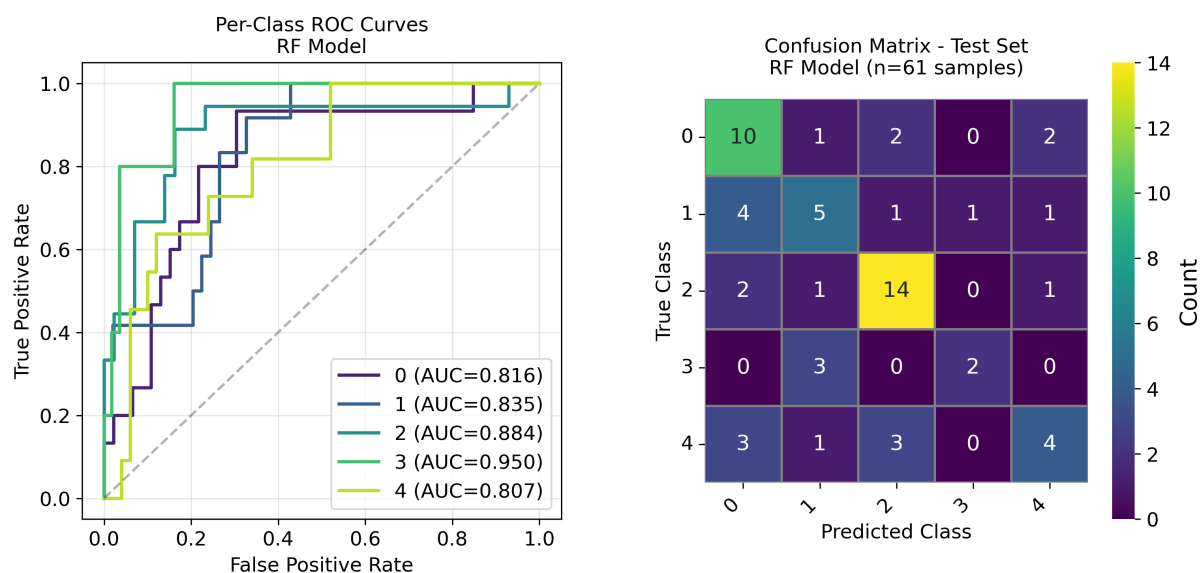


Figure 11.4: Per-class ROC curves (left) and test-set confusion matrix (right) for the final model, fitted on the 16-feature recommended panel and scored on the held-out test set.

11.5 Validation comparison

The validation comparison (Figure 11.5) collects four panels: the three protocols' AUCs per step as grouped bars; feature count against cross-validation AUC; bootstrap AUC with 95% confidence intervals; and a stability panel showing each step's coefficient of variation. It is the visual counterpart to the recommendation logic earlier in this chapter, and the place to look when you want to see why an earlier step was preferred.

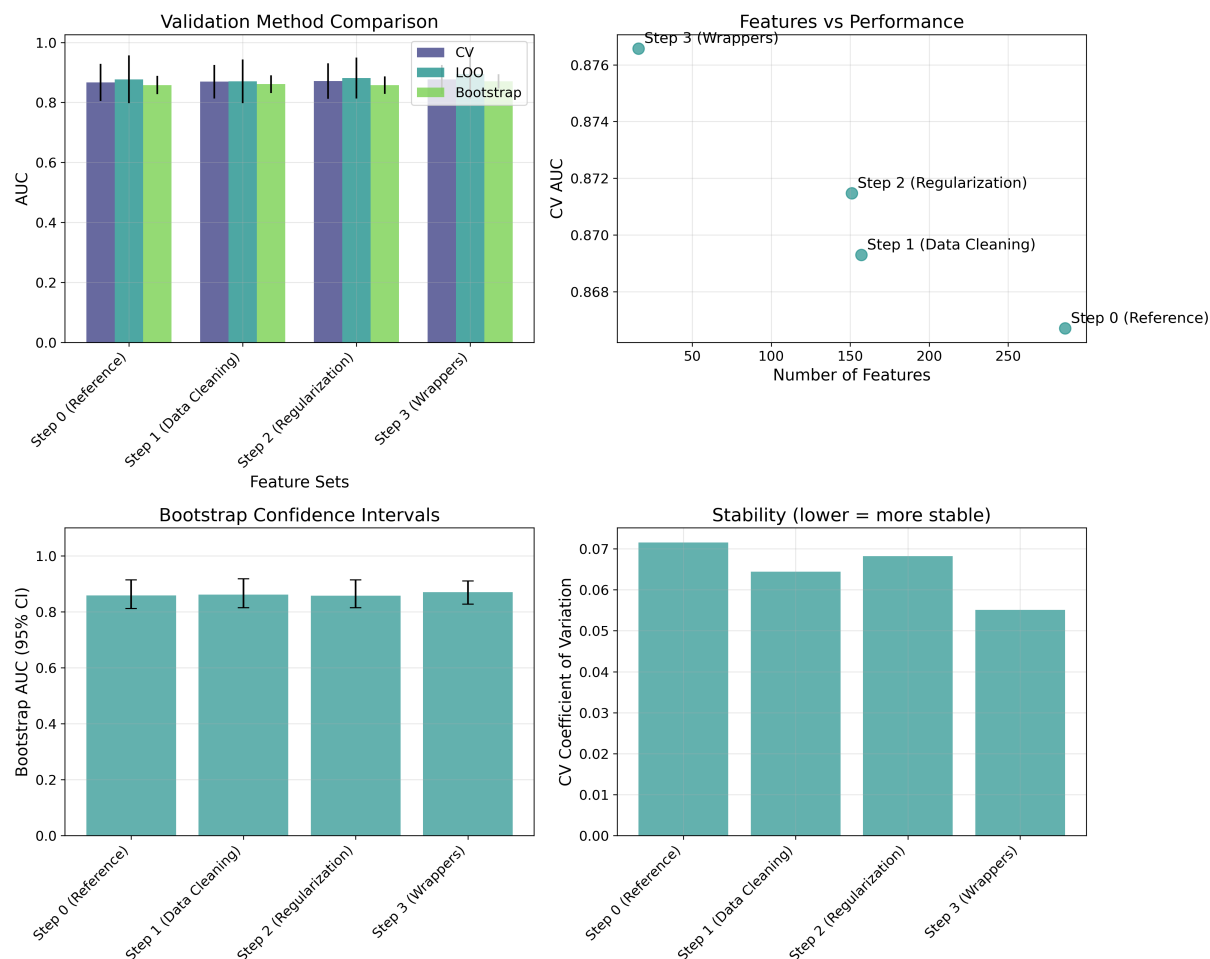


Figure 11.5: Validation comparison across steps. Clockwise from top left: validation method comparison, features versus performance, stability (coefficient of variation, lower is more stable), and bootstrap confidence intervals.

Configuration Reference

Configuration is expressed either as a plain dictionary passed to `SelectOmicsPipeline` or as a `SelectOmicsConfig` object. All values are validated on construction, so an invalid configuration cannot exist in memory. Only `data_path` and `target_column` are required; every other option has a default. Unknown keys are ignored with a `UserWarning`, so configs from older versions remain loadable, but a misspelled key silently reverts to its default: read those warnings.

Appendix A carries the same fields in one table with types and permitted ranges.

12.1 Input and output

Parameter	Default	Description
<code>data_path</code>	required	Path to the input data file.
<code>target_column</code>	required	Name of the class-label column.
<code>schema_version</code>	1.0	Config schema version, for compatibility when loading serialised configs.
<code>input_format</code>	auto	Input format; auto-detected from the extension. One of auto, csv, tsv, excel, parquet, hdf5, feather, json.
<code>hdf5_key</code>	data	Dataset key for HDF5 input.
<code>excel_sheet_name</code>	0	Sheet index or name for Excel input.
<code>output_dir</code>	<code>selectomics_results</code>	Directory for all outputs; created if absent.
<code>output_format</code>	csv	Tabular output format: csv, tsv, excel, parquet, hdf5, feather, json.
<code>excel_output_engine</code>	openpyxl	Excel writer engine (openpyxl or xlsxwriter).
<code>save_intermediate_results</code>	True	Write per-step matrices and summaries; required for resume.
<code>create_visualizations</code>	True	Generate and save all figures.
<code>plot_format</code>	png	Figure format: png, svg, pdf, tiff, jpeg.
<code>verbose</code>	False	Print step-by-step progress through the logger.

12.2 Algorithm, consensus, and compute

Parameter	Default	Description
algorithm	XGB	Classifier: LR, XGB, RF, or SVM. Auto-switched to XGB if the data has missing values and LR or SVM was requested.
n_consensus_models	10	Number of consensus models per step. Higher is more stable but slower.
random_seed	42	Master seed for every stochastic operation.
n_jobs	-1	Threads each estimator may use. Does not change results.
use_gpu	True	Probe for a CUDA GPU for XGBoost; falls back to CPU silently. Does shift results slightly.

12.3 Splitting, cross-validation, and tuning

Parameter	Default	Description
test_size	0.25	Held-out test fraction; $0 < \text{value} < 1$.
cv_splits	None	Explicit fold count; if unset, chosen automatically from the smallest class size.
min_cv_splits	3	Lower bound for automatic fold selection.
max_cv_splits	20	Upper bound for automatic fold selection.
quick_tune_ iterations	50	Randomised-search candidates for the tuning pass.

12.4 Step control and evaluation panels

Parameter	Default	Description
enable_step1	True	Data cleaning.
enable_step2	True	Regularisation.

Parameter	Default	Description
<code>enable_step3</code>	True	Wrappers (auto-skipped below 30 features).
<code>enable_step_evaluations</code>	True	Per-step consensus AUC reports and comparison figures.
<code>enable_final_test_evaluation</code>	True	Held-out test metrics on the recommended panel.

12.5 Selection thresholds and targets

Parameter	Default	Description
<code>max_variance_threshold</code>	0.30	Upper bound of the Step 1 variance grid.
<code>min_correlation_threshold</code>	0.70	Most aggressive (lowest) correlation threshold in the Step 1 grid.
<code>class_aware_correlation</code>	True	Use the pooled within-class correlation matrix with an η^2 priority, rather than total Pearson with a variance priority.
<code>min_features_floor</code>	15	Absolute floor on what any step may return, and in practice the setting that determines panel size.
<code>min_consensus</code>	0.4	The brake on relaxation: how thin agreement may get in pursuit of the floor. None removes the brake.
<code>step2_target_retention</code>	1.0	Fraction of the reachable pool each Step 2 stage keeps.
<code>step2_stage_colsample</code>	0.3	Column-sampling rate for Step 2's XGB stage models, which sets the pool size. Lower widens it.
<code>step2_role</code>	terminal	Whether Step 2 makes the final cut (terminal) or deliberately keeps more for Step 3 (prefilter).
<code>allow_union_rung</code>	False	Add a union rung beneath the intersection ladder in Steps 2 and 3. Trades precision for recall.
<code>stability_threshold</code>	None $\rightarrow$ 0.6	Floor on Step 3 stability selection frequency, below which a feature casts no stability vote.

Parameter	Default	Description
step3_target_retention	0.6	Target fraction of Step 3 input features retained.

12.6 Validation, calibration, nested CV, and ranking

Parameter	Default	Description
n_bootstrap	100	Bootstrap iterations for feature-set validation.
calibrate_probabilities	False	Wrap the final model in isotonic CalibratedClassifierCV.
enable_nested_cv	False	Finish run() with nested CV of the whole procedure, recommendation included.
outer_cv_splits	5	Outer folds when nested CV is enabled.
enable_holdout_ranking	False	Rank the steps on folds held out of the training split rather than on validation that shares its samples with selection.
holdout_ranking_splits	5	Number of ranking folds taken from the training split.

12.7 Serialisation and construction helpers

SelectOmicsConfig can be built directly, from a preset, from a dictionary, or from a file, and can be written back out:

```
from SelectOmics import SelectOmicsConfig

cfg = SelectOmicsConfig("data.csv", "Class", algorithm="RF")
cfg = SelectOmicsConfig.from_dict({"data_path": "data.csv",
                                   "target_column": "Class"})
cfg = SelectOmicsConfig.from_yaml("config.yaml")
cfg = SelectOmicsConfig.from_json("config.json")

cfg.to_json("config.json")           # round-trippable
cfg.to_yaml("config.yaml")
print(cfg.diff_from_defaults())      # only the non-default fields
tmpl = SelectOmicsConfig.template_yaml() # fully commented template
```

`diff_from_defaults()` excludes `data_path`, `target_column` and `schema_version`, and the `repr` of a config shows only those non-default fields, which makes it a compact record of what you actually changed.

Choosing Key Parameters

Most fields can be left alone. Four decide the character of a run.

13.1 `n_consensus_models`

Controls the trade-off between selection stability and runtime. Each consensus model uses a distinct seed derived from `random_seed`; more models reduce the variance of consensus voting.

Dataset size	Recommended	Notes
$n < 50$	1	Only one model is reliable at this scale
$50 \leq n < 150$	3 to 5	<code>suggest()</code> sets 3
$150 \leq n < 500$	5 to 10	<code>suggest()</code> sets 5
$n \geq 500$	10 to 20	<code>suggest()</code> sets 10

13.2 `min_features_floor`

The absolute floor on what any step may return, and in practice the setting that determines panel size. Because each step relaxes its consensus requirement one vote at a time until the floor is met, this is what to change when the pipeline returns too few features.

Measured with every other setting held fixed, over six development scenarios at five seeds:

Floor	Composite	F1	Kuncheva	Runtime
5	0.716	0.624	0.490	162 s
8	0.784	0.714	0.614	160 s
10	0.808	0.745	0.660	159 s
15 (default)	0.820	0.760	0.683	157 s
20	0.819	0.763	0.674	327 s
25	0.820	0.764	0.672	389 s

Monotonic to 15, then flat: 15, 20 and 25 differ by less than 0.001. What separates them is cost. Floors of 20 and above pass enough features to wake Step 3, the most expensive step, for no measurable gain. Lower it when you expect a compact signal, since a floor above the true number of informative features has to pad the panel.

A floor larger than the consensus ceiling cannot be honoured. It is capped at the number of features some model actually selected, and the capping is logged. This is why a floor of 30 can still return 12 features.

13.3 min_consensus

The brake on the relaxation described in Chapter 4. Relaxation stops there even if the feature floor has not been reached.

Value	Meaning
0.4 (default)	Stop before agreement gets thin
0.8	Strong agreement or nothing
1.0	Unanimity or nothing
None	No brake; relax as far as the floor requires

Read the direction carefully

0.4 is *stricter* than no brake, not looser. Without it the relaxation can bottom out at one vote in ten, which costs both accuracy and reproducibility. The response has an interior optimum: composite objective by setting, over 205 configurations on two scenarios, was 0.751 (None), **0.784** (0.4), 0.743 (0.6), 0.599 (0.8) and 0.503 (1.0). A second phase on three scenarios confirmed 0.4 on every scenario individually. Enforcing agreement helps; demanding all of it does not, because at unanimity almost nothing survives.

Setting `min_consensus=1.0` with `min_features_floor=10` says “give me ten features, but only if all models agree on them”. That conflict is reported as `consensus_limited` with both numbers, so it is visible rather than silently resolved. You cannot specify both the panel size and the agreement level and expect both.

13.4 Retention targets

`step2_target_retention` and `step3_target_retention` each set the fraction of a step’s input the step aims to keep. A target is an upper bound, not a promise: a step cannot retain more features than its models could distinguish between, and when that ceiling binds the step logs it and raises a `UserWarning` naming the target and the closest achievable value.

Neither is the main control on panel size

Measured across five scenarios at two seeds, `min_consensus` changed the result in 10 of 10 cells, `min_features_floor` in 7 of 10, and `step2_target_retention` in 2 of 10. If a run is too aggressive or too conservative, reach for `min_consensus` first.

13.5 If you need sensitivity

SELECTOMICS is built to return few features and be right about them: mean precision 0.970 at recall 0.730. If missing a true feature costs you more than carrying a false one, `allow_union_rung=True` trades in the other direction, adding a union rung beneath the intersection ladder in Steps 2 and 3 so the ceiling becomes “kept by some replicate in at least one stage”. Measured over 10 signal-bearing scenarios at five seeds:

Setting	F1	Precision	Recall	Features	Runtime
default	0.849	0.902	0.844	10.8	145 s
allow_union_rung=True	0.755	0.726	0.897	15.4	257 s

Three things to know before using it.

- **Never combine it with `step2_role="prefilter"`.** That pairing measured precision 0.318 and F1 0.411, the worst of the four arms tested. The prefilter role on its own reaches the same recall but takes nine times the runtime.
- **It is a loss in the sparse high- p regime**, which is the regime this package targets. Precision fell from 0.752 to 0.247 at $p = 5000$ with 5 informative features, and from 1.000 to 0.655 at $p = 10\,000$, the latter buying no recall at all because recall was already 1.000. It paid off at $p = 200$ (recall 0.62 to 0.80 at unchanged precision 1.000) and at $p = 1000$ (0.80 to 0.947 for nine precision points).
- **It is less disciplined about noise.** On a null control with no informative features it returned 17.0 false positives against 9.4 for the default.

Everything else is unchanged: the same evaluations run, the same recommendation is made, and the held-out test is spent the same way.

Presets and Suggestions

Presets are classmethod builders that set a coherent group of parameters for a common use case. Override any of them with keyword arguments, and reach all four from the CLI with `-preset`.

Preset	Models	Tune	Boot	Intended use
quick	5	10	10	Rapid iteration; Step 3 off
standard	10	25	50	Most analyses, the default posture
thorough	20	50	100	Publication-quality, runtime no object
omics	10	20	100	Small- n /large- p , benchmark-tuned

```
from SelectOmics import SelectOmicsConfig

cfg = SelectOmicsConfig.standard("data.csv", "Class")
cfg = SelectOmicsConfig.quick("data.csv", "Class", algorithm="RF", verbose=True)
```

14.1 The omics preset

Beyond the counts above it sets `stability_threshold` to 0.259 and `step3_target_retention` to 0.354, the surviving thresholds from a four-phase configuration search on synthetic omics scenarios (winning config `h_071`). Prefer `standard` or `thorough` for large- n or low-dimensional data.

These values are provisional and need re-deriving

The search that produced them tuned ten parameters, five of which have since been shown to have no effect: the four `*_percentile_range` bounds, which it ranked last by influence (Spearman r between -0.038 and 0.045 , p from 0.52 to 0.98), and `consensus_threshold`. A sixth, `step4_target_retention`, belonged to a step that no longer exists. Worse, the parameter that dominates the result, `min_features_floor`, was fixed at its default throughout and never searched at all. The preset's `consensus_threshold=0.442` is deliberately *not* carried over as a `min_consensus` value, since a starting point and a floor are different quantities. Re-run the search with `benchmarks/config_search.py` before relying on these numbers.

14.2 Data-driven suggestions

`SelectOmicsConfig.suggest(data_path, target_column, **overrides)` inspects the data file (shape, missingness, class balance) and returns a starting configuration with a printed rationale. Use it when you are unsure which settings suit an unfamiliar dataset, then review and adjust.

```
import SelectOmics
from SelectOmics import SelectOmicsConfig

SelectOmics.enable_logging()    # so the rationale is printed
cfg = SelectOmicsConfig.suggest("data.csv", "Class")
cfg.to_yaml("my_config.yaml")  # review and edit, then from_yaml() it back
```

What it adjusts: `n_consensus_models` and `n_bootstrap` scaled to sample size; `enable_step3` disabled for $n < 50$ or when fewer than 30 features are present; `quick_tune_iterations` reduced to 30 for $p > 10\,000$; and warnings for severe class imbalance, very small classes, multiclass problems, and extreme n/p . If the data cannot be inspected it falls back to the standard defaults and logs a note. Caller overrides are applied last.

Command-Line Interface

Installing the package provides a `selectomics` command with three subcommands. Global flags `-verbose/-v` (DEBUG logging) and `-version` apply throughout.

15.1 `selectomics suggest`

Inspect a dataset and print (or save) a recommended configuration.

```
selectomics suggest data.csv --target Disease
selectomics suggest data.csv --target Disease --output config.yaml
selectomics suggest data.csv --target Disease --output config.yaml --set
    algorithm=RF
```

15.2 `selectomics template`

Emit a fully commented YAML configuration template, to stdout or a file. Every field appears with its default, valid range, and a description.

```
selectomics template
selectomics template --output my_config.yaml
```

15.3 `selectomics run`

Execute the pipeline. Provide a data file and target, optionally with a preset, or point at a saved configuration file.

```
# From a preset, overriding the output directory
selectomics run data.csv --target Disease --preset omics --output results/

# Override individual fields with repeatable --set
selectomics run data.csv --target Disease --set n_consensus_models=5 \
    --set min_consensus=0.8 \
    --set enable_step3=false

# From a saved config file, resuming an interrupted run
selectomics run --config my_config.yaml --resume
```

Argument	Meaning
<code>data</code>	Input file (required unless <code>-config</code> is given).
<code>-target/-t</code>	Target column (required unless <code>-config</code>).
<code>-preset/-p</code>	quick, standard (default), thorough, or omics.
<code>-config/-c</code>	YAML or JSON config file; overrides <code>-preset</code> .
<code>-output/-o</code>	Override the output directory.
<code>-resume</code>	Resume from an existing checkpoint.
<code>-set KEY=VALUE</code>	Override any field; repeatable; bool/int/float auto-typed.

On completion the command saves results and prints the selected feature count, the recommended panel, the output directory, and the held-out test AUC. It returns exit code 0 on success and 1 on a user error it reports itself, such as a missing data file, an unreadable config, or a target column that is not in the data. An argument the parser rejects outright, such as an unknown `-preset`, exits 2, which is argparse's convention for a usage error.

Output Files

All outputs are written to `output_dir`. Tabular files use `output_format` and figures use `plot_format`; the extensions below show the CSV/PNG defaults.

16.1 Final results

File	Contents
<code>config.json</code>	The serialised pipeline configuration.
<code>recommended_features.csv</code>	The panel to use. What the pipeline evaluation recommends.
<code>selected_features.csv</code>	The last step's feature list, which is the same list unless an earlier step was recommended.
<code>pipeline_summary.csv</code>	Feature count, features removed, cross-validation AUC, and cumulative reduction at each step.
<code>feature_provenance.csv</code>	Per-feature survival across the steps: a boolean per step, the number of steps survived, and the step at which each feature was first dropped.
<code>feature_set_validation.csv</code>	The multi-protocol validation comparison with adequacy flags.
<code>final_<ALGO>_comprehensive_metrics.json</code>	Full held-out test metrics for the recommended panel (Chapter 9). <ALGO> is the algorithm actually used, so a NaN-triggered switch to XGB is visible in the filename.
<code>.selectomics_checkpoint.pkl</code>	Full pipeline state for resume (written after each step).

Two feature lists, and they can differ

`recommended_features` is the deliverable. `selected_features` is whichever step ran last. They are usually identical, and when they are not it is because a later step pruned past the point where it helped, which is exactly the case where reading the wrong file costs you the result.

16.2 Final figures

File	Contents
<code>pipeline_summary.png</code>	Feature count versus AUC across steps.
<code>learning_curve_roc.png</code>	Learning curve and training-versus-test ROC.
<code>roc_confusion.png</code>	Per-class ROC and the test confusion matrix.
<code>validation_comparison.png</code>	The four-panel validation comparison.

16.3 Per-step intermediate files

For each enabled step N , when `save_intermediate_results` is on, the pipeline writes the reduced feature matrix, the dropped-feature lists, a plain-text `stepN_summary.txt`, and (when visualisation is on) the consensus-versus-reference box plot and ROC figures. Step 0 writes `step0-reference_boxplots`, `step0_reference_roc` and `step0_summary.txt`. The step-specific tables are listed alongside each step in Chapter 7, for example `step1_dropped_by_correlation` and `step3_selection_diagnostics`.

Advanced Usage

17.1 Stepwise and chained execution

Every stage is also a public method returning `self`, so the pipeline can be driven step by step or as a fluent chain. This is useful for inspecting intermediate state, for stopping early, or for running a single step on its own.

```
pipeline = selectomics.SelectOmicsPipeline(config)
(pipeline
 .load_data()
 .run_step0_reference()
 .run_step1_cleaning()
 .run_step2_regularization()
 .run_step3_wrapper()
 .validate_features())

# Inspect intermediate feature sets
after_step2 = pipeline.get_selected_features("step2")
provenance = pipeline.get_feature_provenance()
```

`load_data()` must be called before Step 0, and `run_step0_reference()` before Steps 1 to 3; calling out of order raises `RuntimeError`. The full pipeline is the default and the intended use. Running steps separately is supported for inspection and for method development, not as a shortcut past the evaluation: a panel taken from a single step has not been compared against any alternative.

Method	Returns
<code>run(validate=True, resume=False)</code>	The results dict, after the whole pipeline.
<code>get_recommended_features()</code>	The panel the evaluation recommends.
<code>get_selected_features(step=None)</code>	A named step's panel, or the latest.
<code>get_feature_provenance()</code>	Per-feature survival across completed steps.
<code>rank_steps_on_holdout()</code>	Per-step AUC on folds held out of the training split.
<code>run_nested_cv()</code>	Fold-level generalisation of the whole procedure.
<code>save_results(output_dir=None)</code>	None; writes the files above.

The constructor also accepts an `on_step_complete` callback, invoked at the end of each completed step with `(step_key, result_dict)`. Exceptions raised inside the callback are caught and logged, so a faulty callback never aborts a long run.

17.2 Checkpointing and resume

When `save_intermediate_results` is enabled, the full pipeline state is pickled to `.selectomics_checkpoint.pkl` after each step. A subsequent run with `resume=True` (or `-resume` on the CLI) loads the checkpoint and re-executes only the steps after the last completed one. The checkpoint records a fingerprint of the source data, the package version, and the algorithm, so a resume against changed data, an incompatible version, or a different learner is refused rather than silently mixing results from two different runs. Each refusal says which stamp failed and starts the run fresh.

```
pipeline = selectomics.SelectOmicsPipeline(config)
results  = pipeline.run(resume=True)
```

17.3 Nested cross-validation

`enable_nested_cv=True` adds a nested cross-validation pass at the end of `run()`, also callable directly as `run_nested_cv()`. Each outer fold reruns *the configured procedure* on its training portion, selection steps, validation and the recommendation alike, then scores every step's panel on the held-out fold. This is what makes it an estimate of how the whole procedure generalises, step choice included, rather than of one fixed panel.

```
config["enable_nested_cv"] = True
config["outer_cv_splits"]  = 5
results = selectomics.SelectOmicsPipeline(config).run()
nested  = results["nested_cv"]

print(nested["mean_auc"], nested["std_auc"])    # the recommended panel
print(nested["mean_regret"])                    # best-with-hindsight minus chosen
print(nested["feature_stability"])              # how often each feature recurs
```

Alongside `fold_aucs`, `fold_features` and `fold_recommended_steps`, it reports `step_summary`, `last_step_mean_auc`, `best_step_mean_auc` and `mean_regret`, the last being how much held-out AUC the recommendation gave up against the best step chosen with hindsight. A fold whose score could not be computed is reported as NaN rather than scored zero.

What it cost and what it showed

Runtime is roughly `outer_cv_splits` full runs, so it is an occasional check rather than a default. Measured on seven datasets and 35 folds, the recommendation held up: panels 14 to 540 times smaller than the full feature set at a held-out AUC within 0.011 of it, and the best ground-truth F1 on every fold where the truth was known. On a null control it exposed a selection optimism of +0.40 AUC, which is what motivated the guard in Chapter 10.

17.4 Ranking the steps on held-out folds

`rank_steps_on_holdout()` scores every step's panel on `holdout_ranking_splits` folds carved from the training split, so no panel is scored on the rows that selected it, and the test split is untouched. The fold count is lowered to the smallest class size when that is smaller, with a warning. It returns a DataFrame indexed by step with `holdout_auc`, `holdout_std`, `holdout_folds` and `holdout_n_features`.

Set `enable_holdout_ranking=True` to have the recommendation use it. Chapter 10 explains why that is off by default.

17.5 Probability calibration

If downstream use depends on calibrated probabilities rather than only the ranking of predictions, set `calibrate_probabilities=True`. The final model is then wrapped in an isotonic `CalibratedClassifierCV` with an adaptive number of inner folds.

17.6 Programmatic logging

By default the package attaches a null log handler so importing it produces no console output. Call `SelectOmics.enable_logging()` once to route human-readable messages to the console, optionally at a chosen level or through your own handler. `verbose=True` in the config makes the pipeline emit more of them; it does not by itself make any of them visible.

```
import logging
import SelectOmics

SelectOmics.enable_logging("INFO")           # or logging.DEBUG, "WARNING", etc.
SelectOmics.enable_logging(handler=logging.FileHandler("selectomics.log"))
```

Reproducibility

SELECTOMICS is designed for exact reproducibility. Every stochastic operation derives from the single `random_seed`: the train/test split, the tuning search, each consensus model, each bootstrap resample, and each stability subsample. The per-stage seed spacings (Chapter 4) guarantee that no two models across the pipeline accidentally share a seed, and no global NumPy random state is modified, so running SELECTOMICS does not perturb other stochastic code in the same session. Threshold searches use deterministic binary search with a fixed evaluation order, and the reference-model cache is keyed by a stable SHA-256 hash of the sorted feature names and algorithm, so caching never depends on Python’s randomised hashing.

Two settings sit outside that guarantee, in opposite ways. `n_jobs` changes nothing: a run on every core selects exactly what a serial run selects, because XGBoost is pinned to `tree_method='exact'`. `use_gpu` does change results slightly, because the CUDA implementation finds splits differently; leave it off for runs you intend to compare against CPU-generated numbers.

Given the same data, the same configuration and the same value of `use_gpu`, results are identical across runs and machines.

Worked Examples

`examples/` carries twelve notebooks on two TCGA cohorts, glioblastoma (GBM, 244 samples, 5 subtypes) and ovarian carcinoma (OV, 284 samples, 4 subtypes), with four omic layers each plus the merged matrix, indexed in `examples/README.md`. Every notebook enables all evaluations and delivers the recommended panel. Each was executed end to end before shipping and then cleared, so the cells you read are the cells that ran.

The data is not committed. `examples/mlomics_data.py` downloads it from a pinned, checksummed MLOmics revision (CC-BY-4.0) on first use, so a clone stays small and every number below is reproducible from the published source rather than from files that exist only in this repository.

Notebook	Runs in	Read it for
GS-GBM/GBM_Quickstart	4 min	The shortest path from a CSV to a panel
GS-OV/OV_SelectOmics_Showcase	20 min	Every evaluation module, baselines at matched AUC, step ablation, nested CV, checkpoint resume
Ten per-layer notebooks	18 to 102 min	One layer each, identical structure, so layers and cohorts compare directly

Three things the per-layer runs demonstrate that are hard to see from a single dataset:

- **The recommendation overrides the last step when a later step prunes too far.** On OV microRNA, Step 3 cuts Step 2's 115 features to 10 and the recommendation keeps the 115. The run logs that the delivered panel is not the last step's.
- **Step 3 usually does not run.** It skips when fewer than 30 features reach it, which is seven of the ten per-layer notebooks. OV methylation is the near-miss: Step 2 returns 29, one short of the gate, so the most expensive step is skipped on a single feature's margin.
- **The held-out test catches optimism no training-split statistic can.** OV copy-number validated well and scored 0.584 on unseen samples, a gap of 0.226 labelled *poor*. Read the gap, not just the AUC.
- **Runtime follows Step 3, not the feature count.** OV copy-number is the slowest notebook at 102 minutes, because Step 2 hands Step 3 seventy features. The 34 000-feature merged matrices finish sooner, because Step 2 cuts them below the gate and Step 3 never runs. `algorithm="RF"` reduces it further: the quickstart runs the GBM microRNA layer in four minutes.

A fourth point is visible only across the merged notebooks. Given all 34 000 features, both cohorts select almost entirely mRNA: 24 of 25 features for GBM, 15 of 15 for OV. The per-layer notebooks are the control that makes this readable.

Benchmarks

The full benchmark report is `benchmarks/BENCHMARKS.md`, with the harnesses beside it and a reproduction recipe at the end of it. This chapter states the headline result and, more importantly, its limits.

20.1 The claim, measured

Against LASSO, ElasticNet, SHAP-based selection, RF importance and other baselines on synthetic omics scenarios:

Metric	SelectOmics	SHAP	LASSO	ElasticNet
Ground-truth recovery (F1)	0.804	0.577	0.548	
Precision	0.970	0.555	0.501	0.232
Recall	0.730		0.893	1.000
Stability (Kuncheva)	0.690	0.420		
Features returned	8.2	33	38	126
False positives on a null control	5.8	66.8	53.6	180.0
Seconds per trial	96.9	0.8	3.5	

The shape of the win is precision. When SELECTOMICS returns a feature it is almost always a real one, and on a null control with no informative features at all it returns 5.8 false positives against RF importance’s 531.8. The advantage grows with p : F1 0.827 in the target regime and 0.915 at $p \geq 5000$.

20.2 Where it loses, and why that is in this chapter

- **Sensitivity.** Recall 0.730 against LASSO’s 0.893 and ElasticNet’s 1.000. It will miss true features. Chapter 13 gives the flag that trades the other way, and the price.
- **Large n , dense signal.** It ranks fifth of eight there; a default ElasticNet scores 1.000 against its 0.729. This is outside the regime it claims.
- **Runtime.** 124 times SHAP on synthetic data, 29 times LASSO on real integrated data.
- **Stability does not transfer.** Kuncheva 0.690 and first place on synthetic data, but mid-table on real TCGA lower-grade glioma, where ElasticNet and LASSO are more stable on every layer. Parsimony transfers; stability does not.
- **Step 3 fires in 1 of 55 runs.** On wide inputs the shipped pipeline is effectively two steps. Three attempts to change that are documented, all negative.

One line

Choose SELECTOMICS when a short, defensible, reproducible panel is the deliverable. Do not choose it when sensitivity is the objective, when the data is large- n and low-dimensional, or when runtime is binding.

Troubleshooting

Symptom	Explanation and remedy
Algorithm silently became XGB	The data contains missing values and LR/SVM cannot accept them. This is intentional and is logged. Impute beforehand if you must keep LR or SVM.
Step 3 was skipped	Fewer than 30 features reached it. Recursive elimination is not useful on an already-clean panel; this is expected behaviour, not a failure.
The delivered panel is not the last step's	The pipeline evaluation recommended an earlier step. Read <code>validation.recommendation.reason</code> ; this is the system working.
Very few features selected	Raise <code>min_features_floor</code> , which is the control on panel size. <code>min_consensus</code> is the other one, and it also changes how strong a claim the panel carries.
A retention target seems to do nothing	The step could not reach it. A <code>UserWarning</code> names the target and the closest achievable value; the models simply did not use that many features.
<code>agreement_label</code> is minimal	The panel size was reached by relaxing almost all the way. Common on XGB and SVM, whose importance pools are narrow. Not necessarily wrong, but state it alongside the feature list.
Selection optimism suspected	The recommended panel beat the unselected reference by more than 0.05 on the training split. Treat the validation score as unreliable and read the held-out test.
A poor generalisation gap	Validated well, scored badly on unseen samples. Do not use the panel. More data, or fewer selection steps.
Wide bootstrap confidence intervals	The sample is small relative to the task; the validation table flags this. Interpret the selection cautiously.

Symptom	Explanation and remedy
A validation protocol warned about iterations	It completed fewer than half of them. The interval rests on that many samples, not on <code>n_bootstrap</code> .
GPU not used	No CUDA GPU or <code>xgboost</code> below 2.0 was found; the run fell back to CPU. Set <code>use_gpu=False</code> to skip the probe.
Resume did nothing	<code>save_intermediate_results</code> must have been enabled on the earlier run, and the data and package version must match the checkpoint.
Sample-size warnings printed	The dataset triggered one of the four adequacy signals (Chapter 6). These are advisory; the run continues.

Comprehensive Configuration Table

This appendix lists every configuration parameter accepted by `SelectOmicConfig`, with its type, default value, permitted values or range, and a short description. All parameters are validated when the configuration object is constructed, so an invalid value raises an error immediately rather than failing partway through a run. Only `data_path` and `target_column` are required; all others have the defaults shown.

Parameter	Type	Default	Allowed / range	Description
<code>data_path</code>	str/Path	required	existing file	Path to the input data table.
<code>target_column</code>	str	required	column name	Name of the class-label column.
<code>schema_version</code>	str	1.0	1.0	Configuration schema version, for forward compatibility.
<code>input_format</code>	str	auto	auto, csv, tsv, excel, parquet, hdf5, feather, json	Input format; auto-detected from the extension.
<code>hdf5_key</code>	str	data	any key	Dataset key for HDF5 input.
<code>excel_sheet_name</code>	str/int	0	index or name	Sheet for Excel input.
<code>output_dir</code>	str/Path	<code>selectomics_results</code>	any path	Directory for all outputs; created if absent.
<code>output_format</code>	str	csv	csv, tsv, excel, parquet, hdf5, feather, json	Tabular output format.
<code>excel_output_engine</code>	str	openpyxl	openpyxl, xlsxwriter	Excel writer engine.
<code>save_intermediate_results</code>	bool	True	True/False	Write per-step matrices and summaries; required for resume.
<code>create_visualizations</code>	bool	True	True/False	Generate and save all figures.
<code>plot_format</code>	str	png	png, svg, pdf, tiff, jpeg	Figure format.

continued on next page

(Appendix A, continued)

Parameter	Type	Default	Allowed / range	Description
verbose	bool	False	True/False	Print step-by-step progress.
algorithm	str	XGB	LR, XGB, RF, SVM	Classifier; auto-switched to XGB when NaNs are present and LR or SVM was asked for.
n_consensus_models	int	10	≥ 1	Consensus models per step.
random_seed	int	42	$[0, 2^{32})$	Master seed for every stochastic operation.
n_jobs	int	-1	-1 or ≥ 1	Threads per estimator. Does not change results.
use_gpu	bool	True	True/False	Probe for a CUDA GPU for XGBoost. Does shift results slightly.
test_size	float	0.25	$0 < x < 1$	Held-out test fraction.
cv_splits	int/None	None	≥ 2 or None	Explicit fold count; automatic if None.
min_cv_splits	int	3	≥ 2	Lower bound for automatic fold selection.
max_cv_splits	int	20	$\geq$ <code>min_cv_splits</code>	Upper bound for automatic fold selection.
quick_tune_ - iterations	int	50	≥ 1	Randomised-search candidates for tuning.
enable_step1	bool	True	True/False	Data cleaning.
enable_step2	bool	True	True/False	Regularisation.
enable_step3	bool	True	True/False	Wrappers (auto-skipped below 30 features).
enable_step_ - evaluations	bool	True	True/False	Per-step consensus AUC reports and figures.
enable_final_test_ - evaluation	bool	True	True/False	Held-out test metrics on the recommended panel.

continued on next page

(Appendix A, continued)

Parameter	Type	Default	Allowed / range	Description
max_variance_-threshold	float	0.30	> 0	Upper bound of the Step 1 variance grid.
min_correlation_-threshold	float	0.70	$0 < x < 1$	Most aggressive correlation threshold in the Step 1 grid.
class_aware_-correlation	bool	True	True/False	Pooled within-class correlation with an η^2 priority, rather than total Pearson with a variance priority.
min_features_floor	int	15	≥ 1	Absolute floor any step may return; capped at the consensus ceiling.
min_consensus	float/None	0.4	$0 < x \leq 1$ or None	Brake on the relaxation from unanimity.
step2_target_-retention	float	1.0	$0 < x \leq 1$	Fraction of the reachable pool each Step 2 stage keeps.
step2_stage_-colsample	float	0.3	$0 < x \leq 1$	Column-sampling rate for Step 2 XGB stage models. Lower widens the pool.
step2_role	str	terminal	terminal, prefilter	Whether Step 2 makes the final cut or defers it to Step 3.
allow_union_rung	bool	False	True/False	Union rung beneath the intersection ladder. Recall over precision.
stability_-threshold	float/None	None	$0 < x \leq 1$ or None	Floor on Step 3 stability frequency. None resolves to 0.6.
step3_target_-retention	float	0.6	$0 < x < 1$	Target fraction of Step 3 input features retained.
n_bootstrap	int	100	≥ 10	Bootstrap iterations for feature-set validation.

continued on next page

(Appendix A, continued)

Parameter	Type	Default	Allowed / range	Description
<code>calibrate_probabilities</code>	bool	False	True/False	Wrap the final model in isotonic calibration.
<code>enable_nested_cv</code>	bool	False	True/False	Nested CV of the whole procedure, recommendation included.
<code>outer_cv_splits</code>	int	5	≥ 2	Outer folds for nested CV.
<code>enable_holdout_ranking</code>	bool	False	True/False	Rank the steps on folds held out of the training split.
<code>holdout_ranking_splits</code>	int	5	≥ 2	Ranking folds taken from the training split; lowered to the smallest class size when that is smaller.

Preset overrides

The presets in Chapter 14 set groups of these parameters. `quick`, `standard` and `thorough` differ only in `n_consensus_models`, `quick_tune_iterations`, `n_bootstrap` and whether Step 3 runs. `omics` additionally sets `stability_threshold` to 0.259 and `step3_target_retention` to 0.354. Any preset value can itself be overridden with a keyword argument or a `-set` flag.

Fields removed since 0.6.x

`consensus_threshold`, the four `*_percentile_range` tuples, `step4_target_retention`, `enable_step4` and `enable_final_importance_analysis` no longer exist. A config that still sets one of them loads with the usual unknown-key `UserWarning` and the field is ignored. Chapters 4 and 13 explain what replaced the first two; the rest belonged to the explainer step, which was removed.

References

Example datasets. The worked examples and figures in this guide use data from The Cancer Genome Atlas (TCGA), obtained through the NCI Genomic Data Commons (<https://portal.gdc.cancer.gov/>). The Glioblastoma Multiforme (GBM) cohort used for the figures is described in the first reference below.

- The Cancer Genome Atlas Research Network. (2008). Comprehensive genomic characterization defines human glioblastoma genes and core pathways. *Nature*, 455(7216), 1061–1068. doi:10.1038/nature07385
- Varoquaux, G. (2018). Cross-validation failure: small sample sizes lead to large error bars. *NeuroImage*, 180, 68–77. doi:10.1016/j.neuroimage.2017.06.061
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1145.
- Figueroa, R. L., Zeng-Treitler, Q., Kandula, S., and Ngo, L. H. (2012). Predicting sample size required for classification performance. *BMC Medical Informatics and Decision Making*, 12(1), 8. doi:10.1186/1472-6947-12-8
- Meinshausen, N., and Bühlmann, P. (2010). Stability selection. *Journal of the Royal Statistical Society: Series B*, 72(4), 417–473. doi:10.1111/j.1467-9868.2010.00740.x
- Shah, R. D., and Samworth, R. J. (2013). Variable selection with error control: another look at stability selection. *Journal of the Royal Statistical Society: Series B*, 75(1), 55–80. doi:10.1111/j.1467-9868.2012.01043.x
- Kuncheva, L. I. (2007). A stability index for feature selection. *Proceedings of the IASTED International Conference on Artificial Intelligence and Applications*, 390–395.
- Lundberg, S. M., and Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30. arXiv:1705.07874